

Introduction to Artificial Intelligence

CSE 401: Artificial Intelligence

Reference Book

- Artificial Intelligence: A Modern Approach
 - Stuart Russell and Peter Norving

Goals of this Course

- This class is a broad introduction to artificial intelligence (AI)
 - AI is a very broad field with many subareas
 - We will cover many of the primary concepts/ideas

Why AI can change our life.....

- As we begin the new millenium
 - science and technology are changing rapidly
 - “old” sciences such as physics are relatively well-understood
 - computers are ubiquitous
- Grand Challenges in Science and Technology
 - understanding the brain
 - reasoning, cognition, creativity
 - creating intelligent machines
 - is this possible?
 - what are the technical and philosophical challenges?
 - arguably AI poses the most interesting challenges and questions in computer science today

This Lecture

- What is intelligence? What is artificial intelligence?
- A very brief history of AI
 - Modern successes: Stanley the driving robot
- An AI scorecard
 - How much progress has been made in different aspects of AI
- AI in practice
 - Successful applications
- The rational agent view of AI

What is Intelligence?

- Intelligence:
 - “the capacity to learn and solve problems” (Websters dictionary)
 - in particular,
 - *the ability to solve novel problems*
 - *the ability to act rationally*
 - *the ability to act like humans*
- Artificial Intelligence
 - build and understand intelligent entities or agents
 - 2 main approaches: “engineering” versus “cognitive modeling”

What's involved in Intelligence?

- Ability to interact with the real world
 - to perceive, understand, and act
 - e.g., speech recognition and understanding and synthesis
 - e.g., image understanding
 - e.g., ability to take actions, have an effect
- Reasoning and Planning
 - modeling the external world, given input
 - solving new problems, planning, and making decisions
 - ability to deal with unexpected problems, uncertainties
- Learning and Adaptation
 - we are continuously learning and adapting
 - our internal models are always being “updated”
 - e.g., a baby learning to categorize and recognize animals

Academic Disciplines relevant to AI

- Philosophy Logic, methods of reasoning, mind as physical system, foundations of learning, language, rationality.
- Mathematics Formal representation and proof, algorithms, computation, (un)decidability, (in)tractability
- Probability/Statistics modeling uncertainty, learning from data
- Economics utility, decision theory, rational economic agents
- Neuroscience neurons as information processing units.
- Psychology/
Cognitive Science how do people behave, perceive, process cognitive information, represent knowledge.
- Computer engineering building fast computers
- Control theory design systems that maximize an objective function over time
- Linguistics knowledge representation, grammars

History of AI

- 1943: early beginnings
 - McCulloch & Pitts: Boolean circuit model of brain
- 1950: Turing
 - Turing's "Computing Machinery and Intelligence"
- 1956: birth of AI
 - Dartmouth meeting: "Artificial Intelligence" name adopted
- 1950s: initial promise
 - Early AI programs, including
 - Samuel's checkers program
 - Newell & Simon's Logic Theorist
- 1955-65: "great enthusiasm"
 - Newell and Simon: GPS, general problem solver
 - Gelertner: Geometry Theorem Prover
 - McCarthy: invention of LISP

History of AI

- 1966—73: Reality dawns
 - Realization that many AI problems are intractable
 - Limitations of existing neural network methods identified
 - Neural network research almost disappears
- 1969—85: Adding domain knowledge
 - Development of knowledge-based systems
 - Success of rule-based expert systems,
 - E.g., DENDRAL, MYCIN
 - But were brittle and did not scale well in practice
- 1986-- Rise of machine learning
 - Neural networks return to popularity
 - Major advances in machine learning algorithms and applications
- 1990-- Role of uncertainty
 - Bayesian networks as a knowledge representation framework
- 1995-- AI as Science
 - Integration of learning, reasoning, knowledge representation
 - AI methods used in vision, language, data mining, etc

Success Stories

- Deep Blue defeated the reigning **world chess champion** Garry Kasparov in 1997
- AI program proved a mathematical conjecture (Robbins conjecture) unsolved for decades
- During the 1991 Gulf War, US forces deployed **an AI logistics planning and scheduling program** that involved up to 50,000 vehicles, cargo, and people
- NASA's on-board **autonomous planning program** controlled the scheduling of operations for a spacecraft
- `Proverb` solves crossword puzzles better than most humans
- Robot driving: DARPA grand challenge 2003-2007
- 2006: face recognition software available in consumer cameras

Example: DARPA Grand Challenge

- Grand Challenge
 - Cash prizes (\$1 to \$2 million) offered to first robots to complete a long course completely unassisted
 - Stimulates research in vision, robotics, planning, machine learning, reasoning, etc
- 2004 Grand Challenge:
 - 150 mile route in Nevada desert
 - Furthest any robot went was about 7 miles
 - ... but hardest terrain was at the beginning of the course
- 2005 Grand Challenge:
 - 132 mile race
 - Narrow tunnels, winding mountain passes, etc
 - Stanford 1st, CMU 2nd, both finished in about 6 hours
- 2007 Urban Grand Challenge
 - This November in Victorville, California

Stanley Robot

Stanford Racing Team

www.stanfordracing.org



Next few slides courtesy of Prof.
Sebastian Thrun, Stanford University

2004: Barstow, CA, to Primm, NV



150 mile off-road robot race across the Mojave desert

- Natural and manmade hazards
- No driver, no remote control
- No dynamic passing
- Fastest vehicle wins the race (and 2 million dollar prize)

The Grand Challenge Race



HAL: from the movie 2001

- *2001: A Space Odyssey*
 - classic science fiction movie from 1969
- HAL
 - part of the story centers around an intelligent computer called HAL
 - HAL is the “brains” of an intelligent spaceship
 - in the movie, HAL can
 - speak easily with the crew
 - see and understand the emotions of the crew
 - navigate the ship automatically
 - diagnose on-board problems
 - make life-and-death decisions
 - display emotions
- In 1969 this was science fiction: is it still science fiction?



Hal and AI

- *HAL's Legacy: 2001's Computer as Dream and Reality*
 - MIT Press, 1997, David Stork (ed.)
 - discusses
 - HAL as an intelligent computer
 - are the predictions for HAL realizable with AI today?
- Materials online at
 - <http://mitpress.mit.edu/e-books/Hal/contents.html>
- The website contains
 - full text and abstracts of chapters from the book
 - links to related material and AI information
 - sound and images from the film

Consider what might be involved in building a computer like Hal....

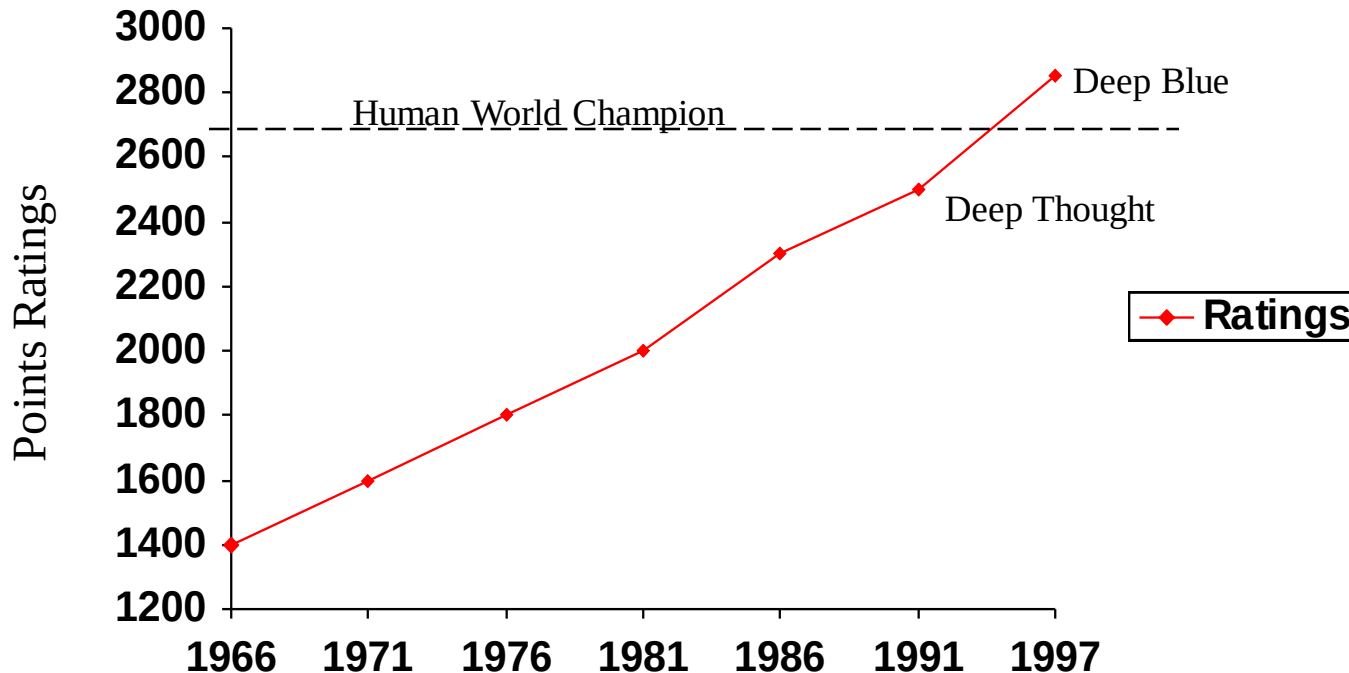
- What are the components that might be useful?
 - Fast hardware?
 - Chess-playing at grandmaster level?
 - Speech interaction?
 - speech synthesis
 - speech recognition
 - speech understanding
 - Image recognition and understanding ?
 - Learning?
 - Planning and decision-making?

Can we build hardware as complex as the brain?

- How complicated is our brain?
 - a neuron, or nerve cell, is the basic information processing unit
 - estimated to be on the order of **10^{12} neurons** in a human brain
 - many more synapses (10^{14}) connecting these neurons
 - cycle time: 10^{-3} seconds (1 millisecond)
- How complex can we make computers?
 - 10^8 or more transistors per CPU
 - supercomputer: hundreds of CPUs, 10^{12} bits of RAM
 - cycle times: order of 10^{-9} seconds
- Conclusion
 - YES: in the near future we can have computers with as many basic processing elements as our brain, but with
 - far fewer interconnections (wires or synapses) than the brain
 - much faster updates than the brain
 - but building hardware is very different from making a computer behave like a brain!

Can Computers beat Humans at Chess?

- Chess Playing is a classic AI problem
 - well-defined problem
 - very complex: difficult for humans to play well



- Conclusion:
 - YES: today's computers can beat even the best human

Can Computers Talk?

- This is known as “speech synthesis”
 - translate text to phonetic form
 - e.g., “fictitious” -> fik-tish-es
 - use pronunciation rules to map phonemes to actual sound
 - e.g., “tish” -> sequence of basic audio sounds
- Difficulties
 - sounds made by this “lookup” approach sound unnatural
 - sounds are not independent
 - e.g., “act” and “action”
 - modern systems (e.g., at AT&T) can handle this pretty well
 - a harder problem is emphasis, emotion, etc
 - humans understand what they are saying
 - machines don’t: so they sound unnatural
- Conclusion:
 - NO, for complete sentences
 - YES, for individual words

Can Computers Recognize Speech?

- Speech Recognition:
 - mapping sounds from a microphone into a list of words
 - classic problem in AI, very difficult
 - “Lets talk about how to wreck a nice beach”
 - (I really said “_____”)
- Recognizing single words from a small vocabulary
 - systems can do this with high accuracy (order of 99%)
 - e.g., directory inquiries
 - limited vocabulary (area codes, city names)
 - computer tries to recognize you first, if unsuccessful hands you over to a human operator
 - saves millions of dollars a year for the phone companies

Recognizing human speech (ctd.)

- Recognizing normal speech is much more difficult
 - speech is continuous: where are the boundaries between words?
 - e.g., “John’s car has a flat tire”
 - large vocabularies
 - can be many thousands of possible words
 - we can use **context** to help figure out what someone said
 - e.g., hypothesize and test
 - try telling a waiter in a restaurant:
“I would like some cream and sugar in my coffee”
 - background noise, other speakers, accents, clogs, etc
 - on normal speech, modern systems are only about 60-70% accurate
- Conclusion:
 - NO, normal speech is too complex to accurately recognize
 - YES, for restricted problems (small vocabulary, single speaker)

Can Computers Understand speech?

- Understanding is different to recognition:
 - “Time flies like an arrow”
 - assume the computer can recognize all the words
 - how many different interpretations are there?

Can Computers Understand speech?

- Understanding is different to recognition:
 - “Time flies like an arrow”
 - assume the computer can recognize all the words
 - how many different interpretations are there?
 - 1. time passes quickly like an arrow?
 - 2. command: time the flies the way an arrow times the flies
 - 3. command: only time those flies which are like an arrow
 - 4. “time-flies” are fond of arrows

Can Computers Understand speech?

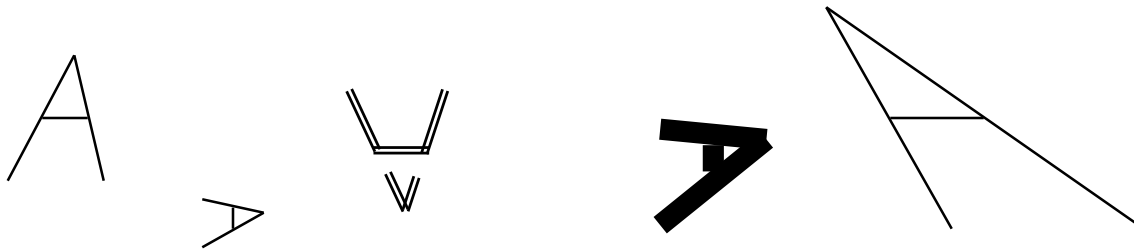
- Understanding is different to recognition:
 - “Time flies like an arrow”
 - assume the computer can recognize all the words
 - how many different interpretations are there?
 - 1. time passes quickly like an arrow?
 - 2. command: time the flies the way an arrow times the flies
 - 3. command: only time those flies which are like an arrow
 - 4. “time-flies” are fond of arrows
 - only 1. makes any sense,
 - but how could a computer figure this out?
 - clearly humans use a lot of implicit commonsense knowledge in communication
- Conclusion: NO, much of what we say is beyond the capabilities of a computer to understand at present

Can Computers Learn and Adapt ?

- Learning and Adaptation
 - consider a computer learning to drive on the freeway
 - we could teach it lots of rules about what to do
 - or we could let it drive and steer it back on course when it heads for the embankment
 - systems like this are under development (e.g., Daimler Benz)
 - e.g., RALPH at CMU
 - in mid 90's it drove 98% of the way from Pittsburgh to San Diego without any human assistance
 - **machine learning** allows computers to learn to do things without explicit programming
 - many successful applications:
 - requires some "set-up": does not mean your PC can learn to forecast the stock market or become a brain surgeon
- Conclusion: YES, computers can learn and adapt, when presented with information in the appropriate way

Can Computers “see”?

- Recognition v. Understanding (like Speech)
 - Recognition and Understanding of Objects in a scene
 - look around this room
 - you can effortlessly recognize objects
 - human brain can map 2d visual image to 3d “map”
- Why is visual recognition a hard problem?



- Conclusion:
 - mostly NO: computers can only “see” certain types of objects under limited circumstances
 - YES for certain constrained problems (e.g., face recognition)

Can computers plan and make optimal decisions?

- Intelligence
 - involves solving problems and making decisions and plans
 - e.g., you want to take a holiday in Brazil
 - you need to decide on dates, flights
 - you need to get to the airport, etc
 - involves a sequence of decisions, plans, and actions
- What makes planning hard?
 - the world is not predictable:
 - your flight is canceled or there's a backup on the 405
 - there are a potentially huge number of details
 - do you consider all flights? all dates?
 - no: commonsense constrains your solutions
 - AI systems are only successful in constrained planning problems
- Conclusion: NO, real-world planning and decision-making is still beyond the capabilities of modern computers
 - exception: very well-defined, constrained problems

Summary of State of AI Systems in Practice

- **Speech synthesis, recognition and understanding**
 - very useful for limited vocabulary applications
 - unconstrained speech understanding is still too hard
- **Computer vision**
 - works for constrained problems (hand-written zip-codes)
 - understanding real-world, natural scenes is still too hard
- **Learning**
 - adaptive systems are used in many applications: have their limits
- **Planning and Reasoning**
 - only works for constrained problems: e.g., chess
 - real-world is too complex for general systems
- **Overall:**
 - many components of intelligent systems are “doable”
 - there are many interesting research problems remaining

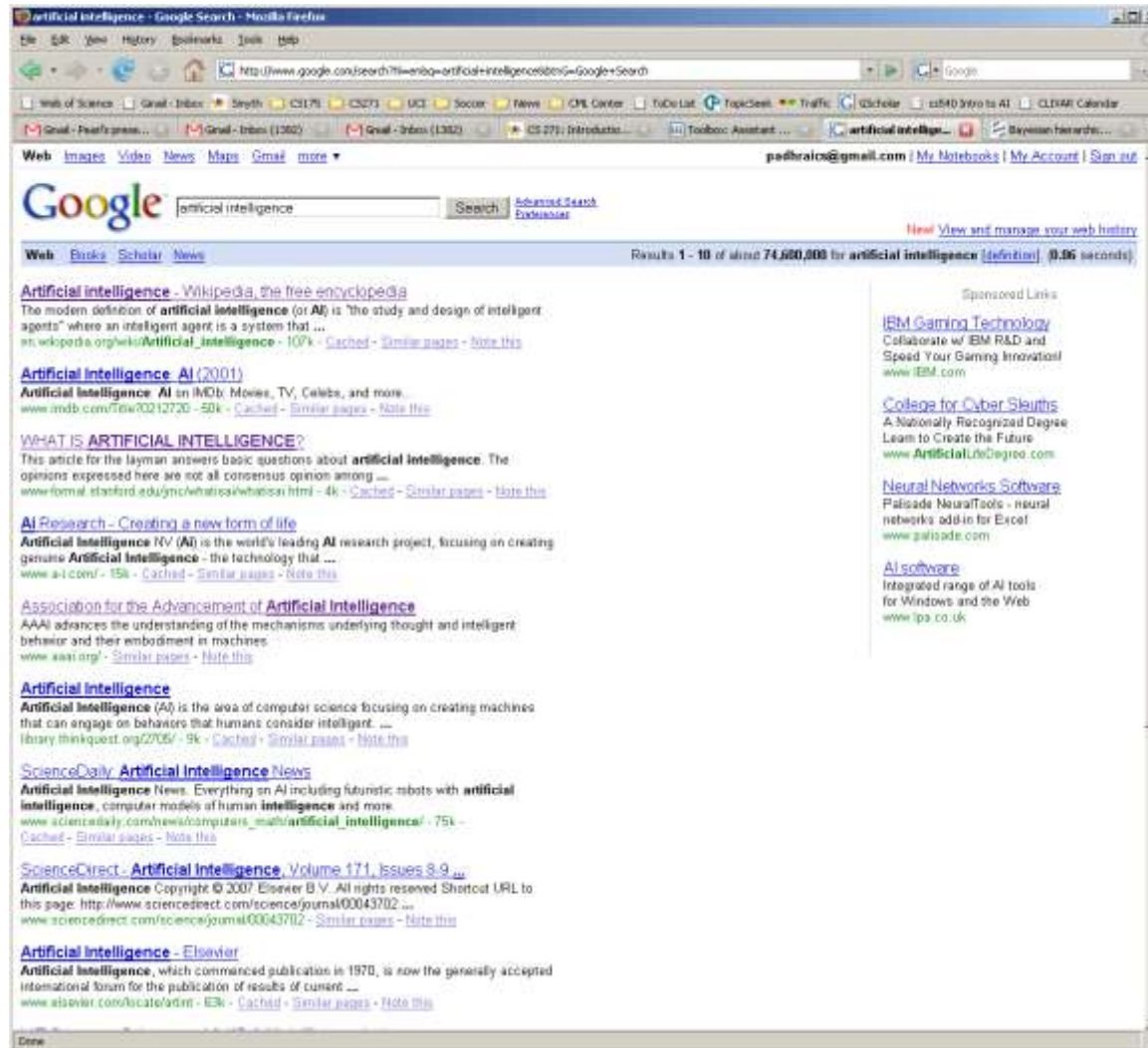
Intelligent Systems in Your Everyday Life

- Post Office
 - automatic address recognition and sorting of mail
- Banks
 - automatic check readers, signature verification systems
 - automated loan application classification
- Customer Service
 - automatic voice recognition
- The Web
 - Identifying your age, gender, location, from your Web surfing
 - Automated fraud detection
- Digital Cameras
 - Automated face detection and focusing
- Computer Games
 - Intelligent characters/agents

AI Applications: Machine Translation

- Language problems in international business
 - e.g., at a meeting of Japanese, Korean, Vietnamese and Swedish investors, no common language
 - or: you are shipping your software manuals to 127 countries
 - solution; hire translators to translate
 - would be much cheaper if a machine could do this
- How hard is automated translation
 - very difficult! e.g., English to Russian
 - “The spirit is willing but the flesh is weak” (English)
 - “the vodka is good but the meat is rotten” (Russian)
 - not only must the words be translated, but their meaning also!
 - is this problem “AI-complete”?
- Nonetheless....
 - commercial systems can do a lot of the work very well (e.g., restricted vocabularies in software documentation)
 - algorithms which combine dictionaries, grammar models, etc.
 - Recent progress using “black-box” machine learning techniques

AI and Web Search



What's involved in Intelligence? (again)

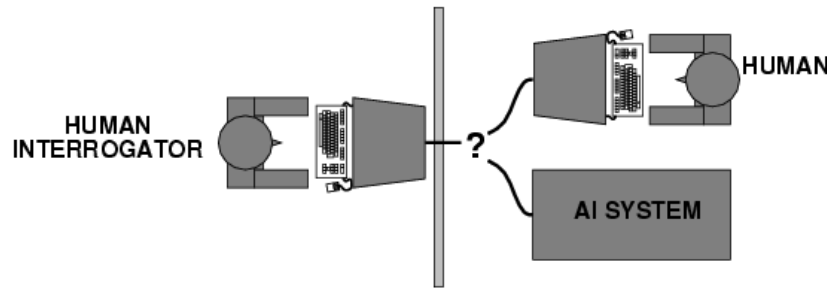
- Perceiving, recognizing, understanding the real world
 - Reasoning and planning about the external world
 - Learning and adaptation
-
- So what general principles should we use to achieve these goals?

Different Types of Artificial Intelligence

1. Modeling exactly how humans actually think
 - Modeling exactly how humans actually act
 - Modeling how ideal agents “should think”
 - Modeling how ideal agents “should act”
- Modern AI focuses on the last definition
 - we will also focus on this “engineering” approach
 - success is judged by how well the agent performs

Acting humanly: Turing test

- Turing (1950) "Computing machinery and intelligence"
- "Can machines think?" → "Can machines behave intelligently?"
- Operational test for intelligent behavior: the Imitation Game



- Suggests major components required for AI:
 - knowledge representation
 - reasoning,
 - language/image understanding,
 - learning
- * Question: is it important that an intelligent system act like a human?

Thinking humanly

- Cognitive Science approach
 - Try to get “inside” our minds
 - E.g., conduct experiments with people to try to “reverse-engineer” how we reason, learning, remember, predict
- Problems
 - Humans don’t behave rationally
 - e.g., insurance
 - The reverse engineering is very hard to do
 - The brain’s hardware is very different to a computer program

Thinking rationally

- Represent facts about the world via logic
- Use logical inference as a basis for reasoning about these facts
- Can be a very useful approach to AI
 - E.g., theorem-provers
- Limitations
 - Does not account for an agent's uncertainty about the world
 - E.g., difficult to couple to vision or speech systems
 - Has no way to represent goals, costs, etc (important aspects of real-world environments)

Acting rationally

- Decision theory/Economics
 - Set of future states of the world
 - Set of possible actions an agent can take
 - Utility = gain to an agent for each action/state pair
 - An agent acts rationally if it selects the action that maximizes its “utility”
 - Or expected utility if there is uncertainty
- Emphasis is on autonomous agents that behave rationally (make the best predictions, take the best actions)
 - on average over time
 - within computational limitations (“bounded rationality”)

Summary of Today's Lecture

- Artificial Intelligence involves the study of:
 - automated recognition and understanding of signals
 - reasoning, planning, and decision-making
 - learning and adaptation
- AI has made substantial progress in
 - recognition and learning
 - some planning and reasoning problems
 - ...but many open research problems
- AI Applications
 - improvements in hardware and algorithms => AI applications in industry, finance, medicine, and science.
- Rational agent view of AI

<https://www.youtube.com/watch?v=IBYRixhYpck>

Chapter 2

Intelligent Agents

Agents

- An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators

Human agent:

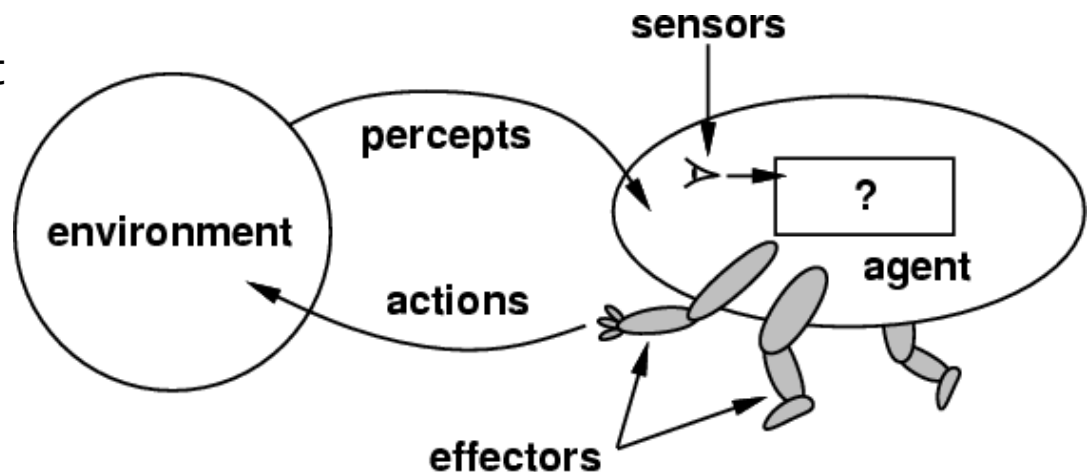
eyes, ears, and other organs for sensors;
hands, legs, mouth, and other body parts for
actuators

Robotic agent:

cameras and infrared range finders for sensors; various motors
for actuators

How to design an intelligent agent?

- An **intelligent agent** perceives its environment via **sensors** and acts rationally upon that environment with its **effectors**.
- A discrete agent receives **percepts** one at a time, and maps this percept sequence to a sequence of discrete **actions**.
- Properties
 - Autonomous
 - Reactive to the environment
 - Pro-active (goal-directed)
 - Interacts with other agents via the environment

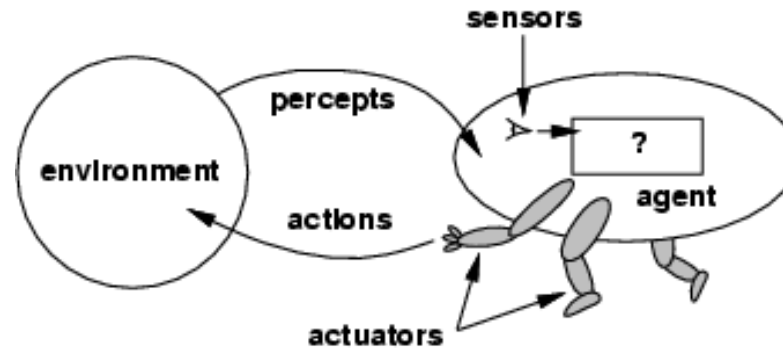


Sensors/percepts and effectors/actions?

- Humans

- Sensors: Eyes (vision), ears (hearing), skin (touch), tongue (gustation), nose (olfaction), neuromuscular system (proprioception)
- Percepts:
 - At the lowest level – electrical signals from these sensors
 - After preprocessing – objects in the visual field (location, textures, colors, ...), auditory streams (pitch, loudness, direction), ...
- Effectors: limbs, digits, eyes, tongue, ...
- Actions: lift a finger, turn left, walk, run, carry an object, ...

Agents and environments

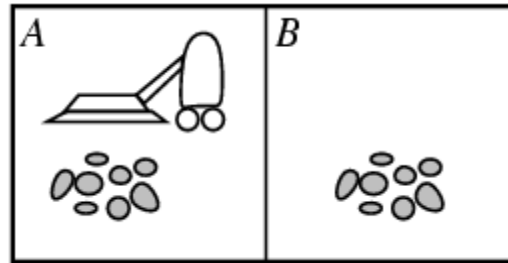


- The agent function maps from percept histories to actions:

$$[f: \mathcal{P}^* \rightarrow \mathcal{A}]$$

- The agent program runs on the physical architecture to produce f
- agent = architecture + program

Vacuum-cleaner world



- Percepts: location and state of the environment, e.g., [A,Dirty], [A,Clean], [B,Dirty]
- Actions: *Left*, *Right*, *Suck*, *NoOp*

Rational agents

- **Performance measure:** An objective criterion for success of an agent's behavior, e.g.,
 - Robot driver?
 - Chess-playing program?
 - Spam email classifier?
- **Rational Agent:** selects actions that is *expected* to maximize its performance measure,
 - given percept sequence
 - given agent's built-in knowledge
 - sidepoint: how to maximize expected future performance, given only historical data

Rational agents

- Rational Agent → Always try to maximize performance.
- No Agent is **Omniscience**. Rationality is distinct from omniscience (all-knowing with infinite knowledge)
- Agents can perform actions in order to modify future percepts so as to obtain useful information (information gathering, exploration)
- An agent is **autonomous** if its behavior is determined by its own percepts & experience (with ability to learn and adapt) without depending solely on built-in knowledge
- To survive, agents must have:
 - Enough built-in knowledge to survive.
 - The ability to learn

Task Environment

- Before we design an intelligent agent, we must specify its “task environment”:

PEAS:

Performance measure

Environment

Actuators

Sensors

DARPA Robotics Challenge

- A prize competition funded by the US Defense Advanced Research Projects Agency, held from 2012 to 2015
- Aimed to develop semi-autonomous ground robots that could do complex tasks in dangerous, degraded, human-engineered environments

DARPA Robotics Challenge

The initial task requirements for robot entries are

- Drive a utility vehicle at the site
- Travel dismounted across rubble
- Remove debris blocking an entryway
- Open a door and enter a building
- Climb an industrial ladder and traverse an industrial walkway
- Use a tool to break through a concrete panel
- Locate and close a valve near a leaking pipe
- Connect a fire hose to a standpipe and turn on a valve

PEAS

- Example: Agent = robot driver in DARPA Challenge
 - Performance measure:
 - Time to complete course
 - Environment:
 - Roads, other traffic, obstacles
 - Actuators:
 - Steering wheel, accelerator, brake, signal, horn
 - Sensors:
 - Optical cameras, lasers, sonar, accelerometer, speedometer, GPS, odometer, engine sensors,

PEAS

- Example: Agent = Medical diagnosis system

Performance measure:

Healthy patient, minimize costs, lawsuits

Environment:

Patient, hospital, staff

Actuators:

Screen display (questions, tests, diagnoses, treatments, referrals)

Sensors:

Keyboard (entry of symptoms, findings, patient's answers)

Environment types

- **Fully observable** (vs. **partially observable**):
 - An agent's sensors give it access to the complete state of the environment at each point in time.
- **Deterministic** (vs. **stochastic**):
 - The next state of the environment is completely determined by the current state and the action executed by the agent.
 - If the environment is deterministic except for the actions of other agents, then the environment is **strategic**
 - Deterministic environments can appear stochastic to an agent (e.g., when only partially observable)
- **Episodic** (vs. **sequential**):
 - An agent's action is divided into atomic episodes. Decisions do not depend on previous decisions/actions.

Environment types

- **Static** (vs. **dynamic**):
 - The environment is unchanged while an agent is deliberating.
 - The environment is **semidynamic** if the environment itself does not change with the passage of time but the agent's performance score does
- **Discrete** (vs. **continuous**):
 - A discrete set of distinct, clearly defined percepts and actions.
 - How we **represent** or **abstract** or **model** the world
- **Single agent** (vs. **multi-agent**):
 - An agent operating by itself in an environment. Does the other agent interfere with my performance measure?

Characteristics of environments

	Fully observable?	Deterministic	Episodic	Static	Discrete?	Single agent?
Solitaire						
Driving						
Internet shopping						
Medical diagnosis						

Characteristics of environments

	Fully observable?	Deterministic?	Episodic?	Static?	Discrete?	Single agent?
Solitaire	No	Yes	Yes	Yes	Yes	Yes
Driving	No	No	No	No	No	No
Internet shopping						
Medical diagnosis						

Characteristics of environments

	Fully observable?	Deterministic?	Episodic?	Static?	Discrete?	Single agent?
Solitaire	No	Yes	Yes	Yes	Yes	Yes
Driving	No	No	No	No	No	No
Internet shopping	No	No	No	No	Yes	No
Medical diagnosis						

Characteristics of environments

	Fully observable?	Deterministic?	Episodic?	Static?	Discrete?	Single agent?
Solitaire	No	Yes	Yes	Yes	Yes	Yes
Driving	No	No	No	No	No	No
Internet shopping	No	No	No	No	Yes	No
Medical diagnosis	No	No	No	No	No	Yes

→ Lots of real-world domains fall into the hardest case!

task environm.	observabl e	deterministic / stochastic	episodic/ sequential	static/ dynamic	discrete/ continuous	agents
crossword puzzle	fully	determ.	sequential	static	discrete	single
chess with clock	fully	strategic	sequential	semi	discrete	multi
poker						
taxi driving	partial	stochastic	sequential	dynamic	continuous	multi
medical diagnosis						
image analysis	fully	determ.	episodic	semi	continuous	single
partpicking robot	partial	stochastic	episodic	dynamic	continuous	single
refinery controller	partial	stochastic	sequential	dynamic	continuous	single
interact. tutor	partial	stochastic	sequential	dynamic	discrete	multi

What is the environment for the DARPA Challenge?

- Agent = robotic vehicle
- Environment = 130-mile route through desert
 - Observable?
 - Deterministic?
 - Episodic?
 - Static?
 - Discrete?
 - Agents?

Agent types

- Five basic types in order of increasing generality:
 - Table Driven agent
 - Simple reflex agents
 - Model-based reflex agents
 - Goal-based agents
 - Problem-solving agents
 - Utility-based agents
 - Can distinguish between different goals
 - Learning agents

Some agent types

- **Table-driven agents**
 - use a percept sequence/action table in memory to find the next action. They are implemented by a (large) **lookup table**. It is not autonomous.
- **Simple reflex agents**
 - are based on **condition-action rules**, implemented with an appropriate production system. They are stateless devices which do not have memory of past world states. It can not save history.
- **Agents with memory**
 - have **internal state**, which is used to keep track of past states of the world.
- **Agents with goals**
 - are agents that, in addition to state information, have **goal information** that describes desirable situations. Agents of this kind take future events into consideration. **Never thinks about cost.**
- **Utility-based agents**
 - base their decisions on **classic axiomatic utility theory** in order to act rationally. **Always thinks about cost.**

Table-driven/reflex agent Architecture

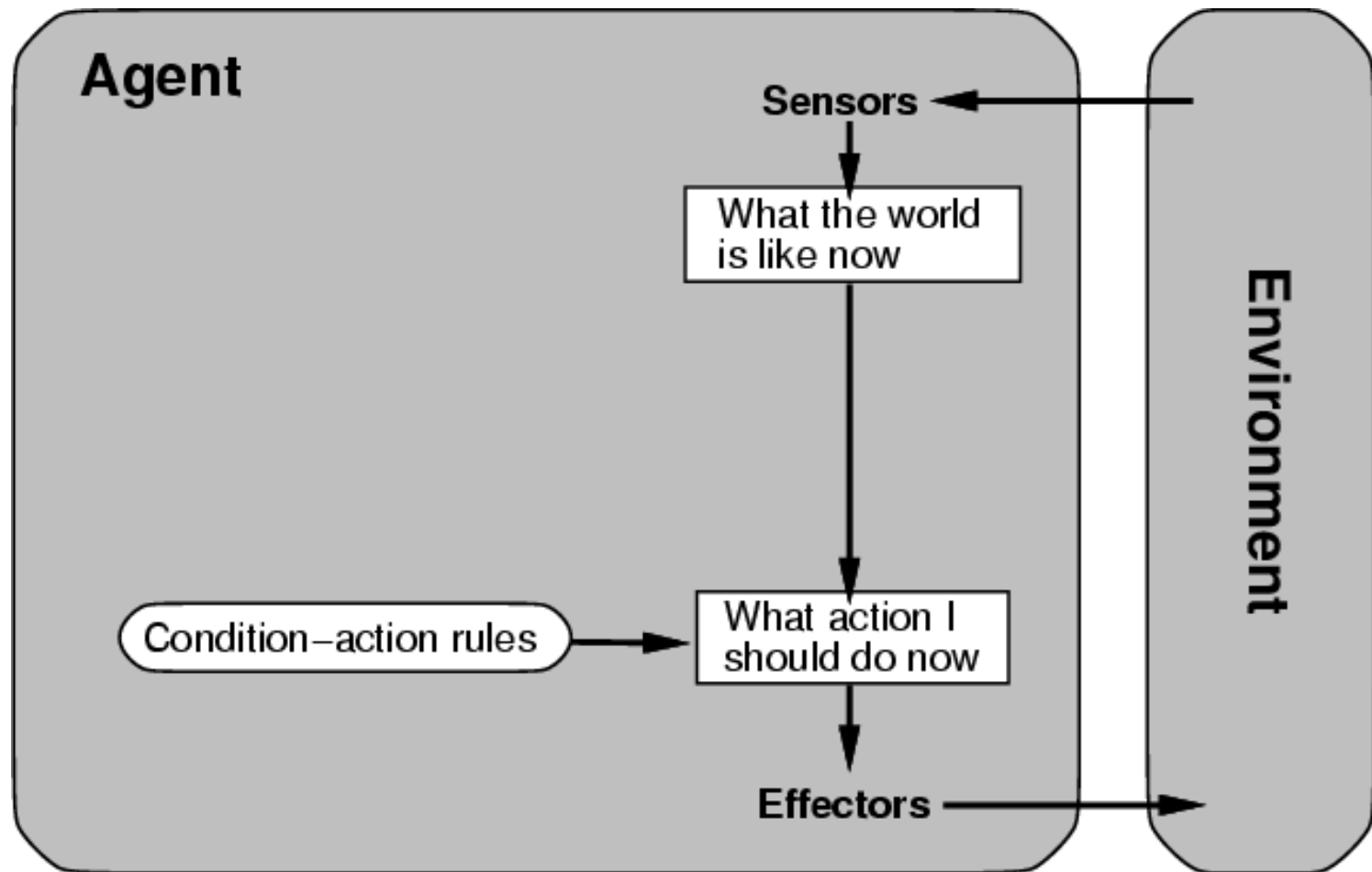


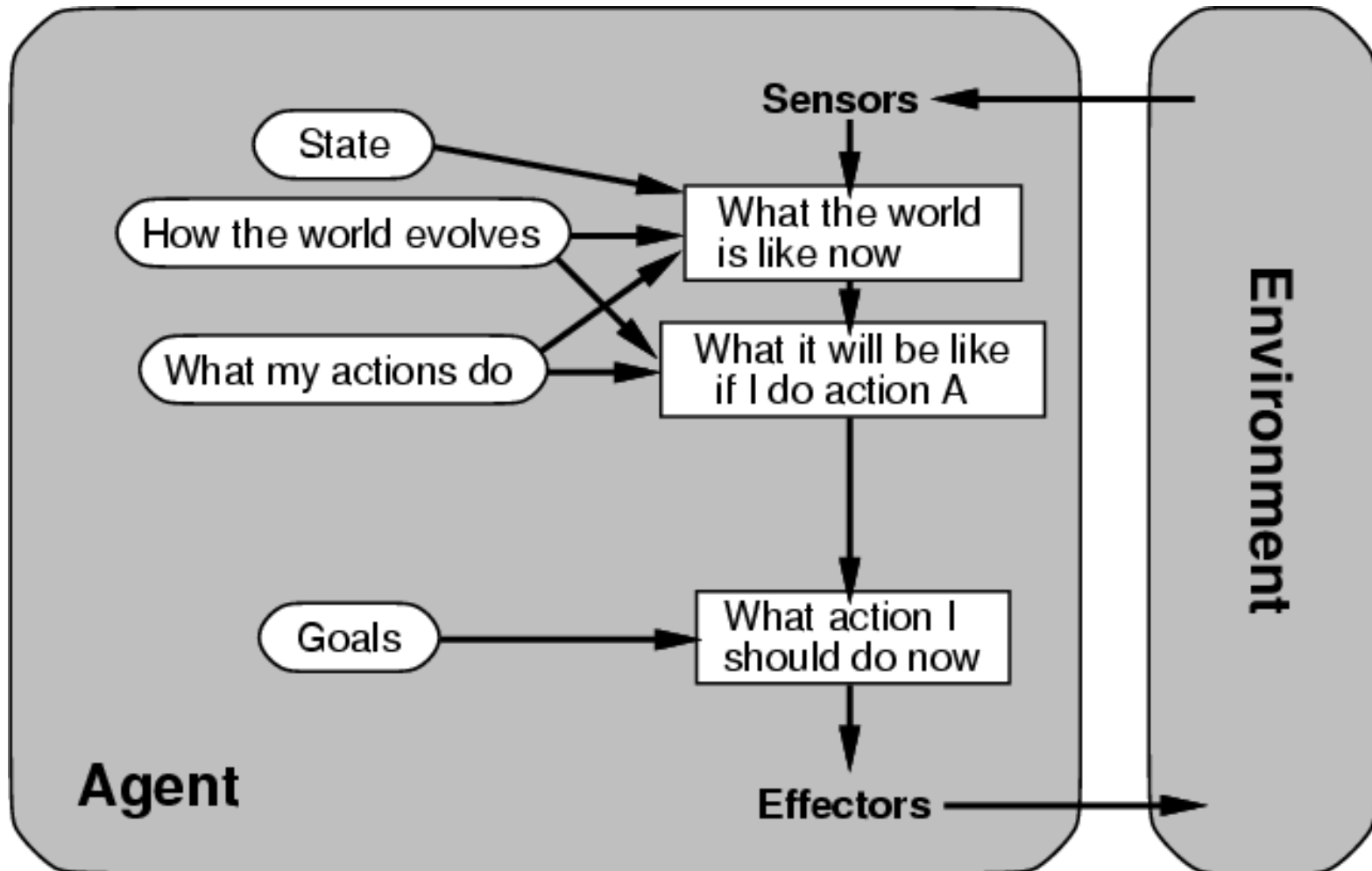
Table-driven agents

- **Table lookup** of percept-action pairs mapping from every possible perceived state to the optimal action for that state
- **Problems**
 - Too big to generate and to store (Chess has about 10^{120} states, for example)
 - No knowledge of non-perceptual parts of the current state
 - Not adaptive to changes in the environment; requires entire table to be updated if changes occur
 - Looping: Can't make actions conditional on previous actions/states

Simple reflex agents

- **Rule-based reasoning** to map from percepts to optimal action; each rule handles a collection of perceived states
- **Problems**
 - Still usually too big to generate and to store
 - Still not adaptive to changes in the environment; requires collection of rules to be updated if changes occur
 - Still can't make actions conditional on previous state

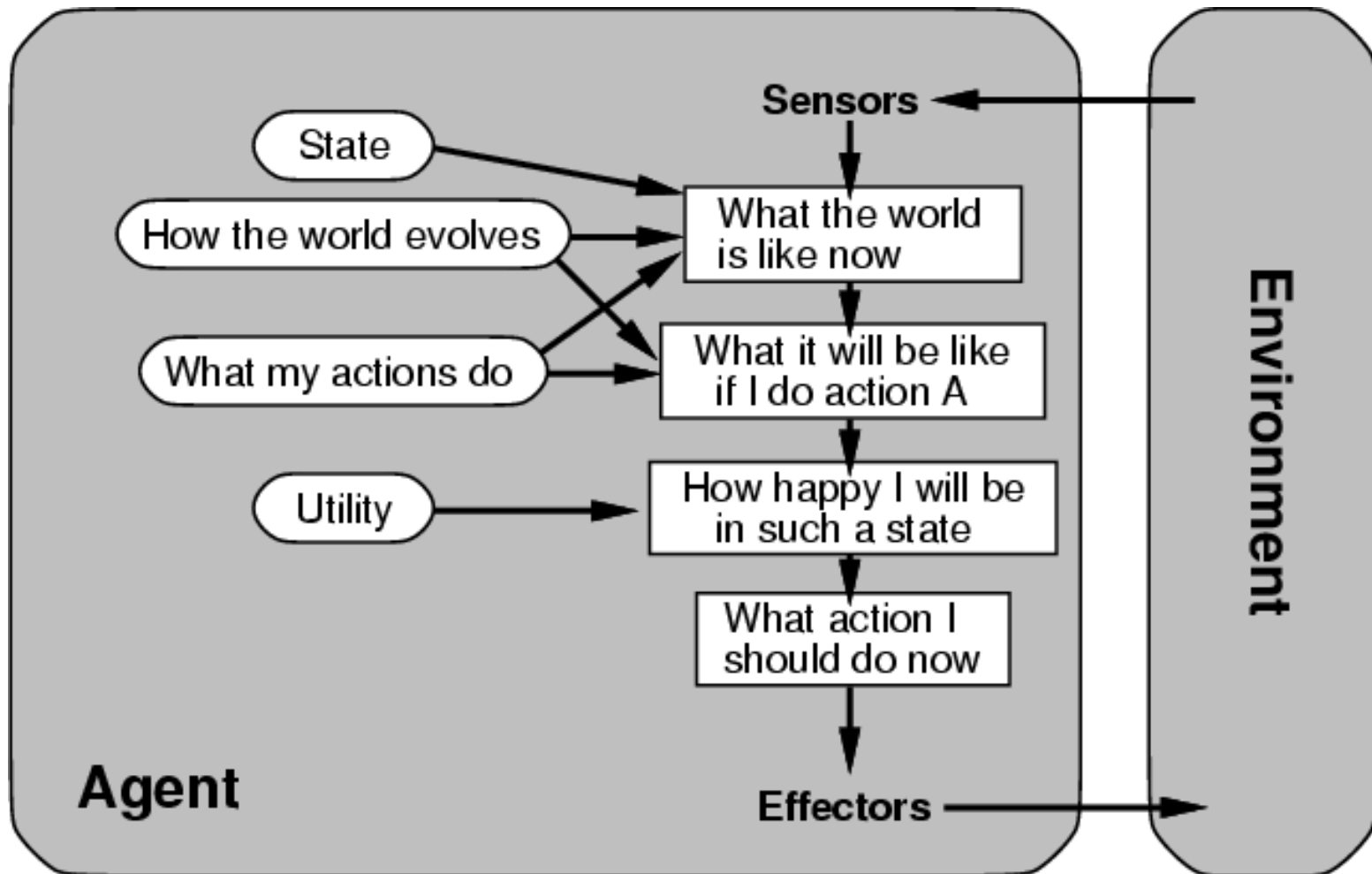
Goal-based agent: Architecture



Goal-based agents

- Choose actions so as to achieve a (given or computed) goal.
- A goal is a description of a desirable situation.
- Keeping track of the current state is often not enough – need to add goals to decide which situations are good
- **Deliberative** instead of **reactive**.
- May have to consider long sequences of possible actions before deciding if goal is achieved – involves consideration of the future, *"what will happen if I do...?"*

Complete utility-based agent



Utility-based agents

- When there are multiple possible alternatives, how to decide which one is best?
- A goal specifies a crude distinction between a happy and unhappy state, but often need a more general performance measure that describes “degree of happiness.”
- Utility function **$U: \text{State} \rightarrow \text{Reals}$** indicating a measure of success or happiness when at a given state.

Summary

- An **agent** perceives and acts in an environment, has an architecture, and is implemented by an agent program.
- An **ideal agent** always chooses the action which maximizes its expected performance, given its percept sequence so far.
- An **autonomous agent** uses its own experience rather than built-in knowledge of the environment by the designer.

Summary (Contd.)

- An **agent program** maps from percept to action and updates its internal state.
 - **Reflex agents** respond immediately to percepts.
 - **Goal-based agents** act in order to achieve their goal(s).
 - **Utility-based agents** maximize their own utility function.
- **Representing knowledge** is important for successful agent design.
- The most challenging environments are
 - partially observable
 - stochastic
 - sequential
 - Dynamic
 - continuous
 - contain multiple intelligent agents

Chapter 3

Problem Solving By Searching

Problem-Solving Agents

- Intelligent agents can solve problems by searching a state-space
- State-space Model
 - the agent's model of the world
 - usually a set of discrete states
 - e.g., in driving, the states in the model could be towns/cities
- Goal State(s)
 - a goal is defined as a desirable state for an agent
 - there may be many states which satisfy the goal test
 - e.g., drive to a town with a ski-resort
 - or just one state which satisfies the goal
 - e.g., drive to Mammoth
- Operators (actions, successor function)
 - operators are legal actions which the agent can take to move from one state to another

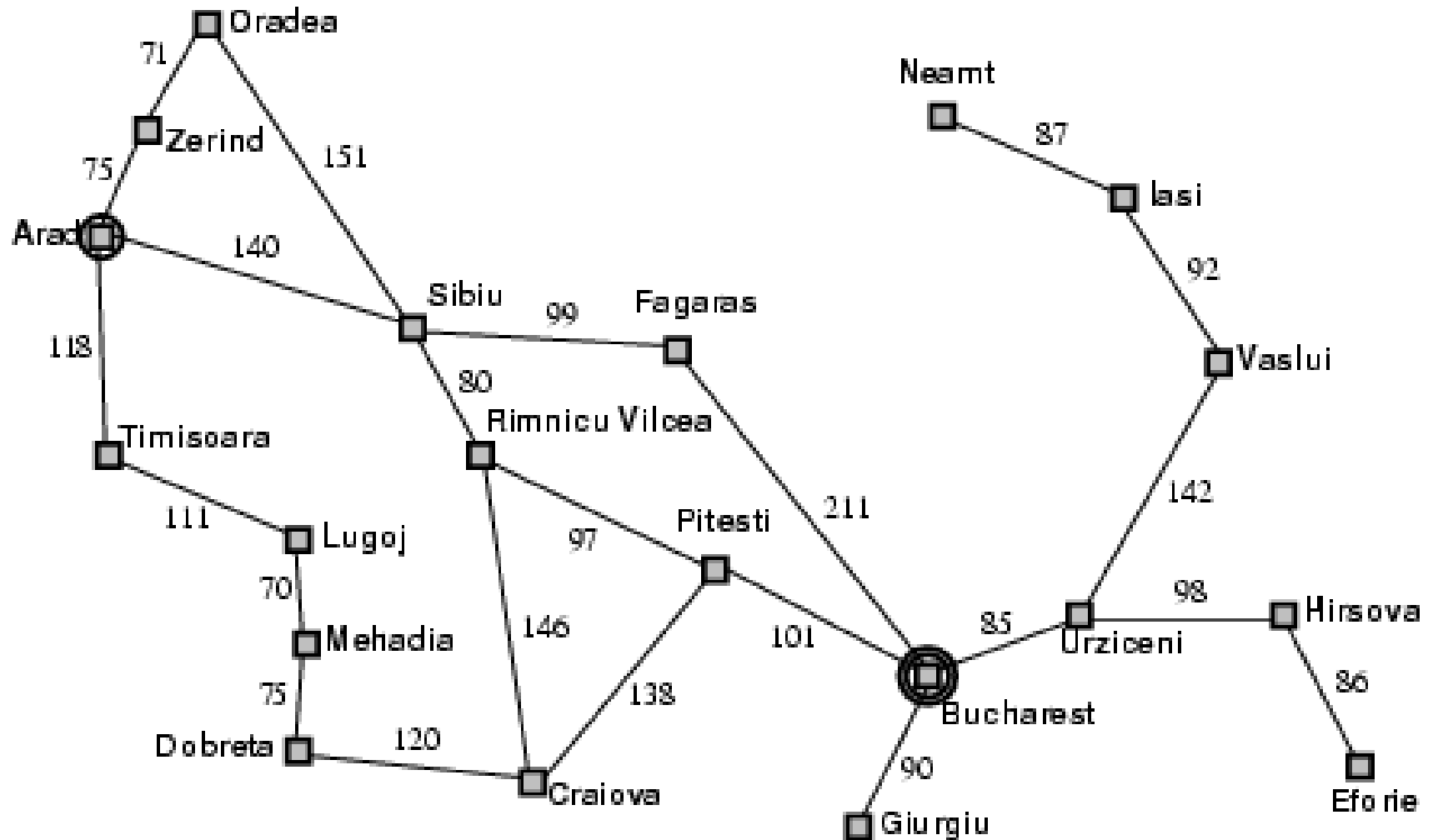
Initial Simplifying Assumptions

- Environment is static
 - no changes in environment while problem is being solved
- Environment is observable
- Environment and actions are discrete
 - (typically assumed, but we will see some exceptions)
- Environment is deterministic

Example: Traveling in Romania

- On holiday in Romania; currently in Arad
- Flight leaves tomorrow from Bucharest
- Formulate goal:
 - be in Bucharest
- Formulate problem:
 - **states**: various cities
 - **actions/operators**: drive between cities
- Find solution
 - By searching through states to find a goal
 - sequence of cities, e.g., Arad, Sibiu, Fagaras, Bucharest
- Execute states that lead to a solution

Example: Traveling in Romania



State-Space Problem Formulation

A **problem** is defined by four items:

1. **initial state** e.g., "at Arad"

2. **actions** or **successor function**

$S(x)$ = set of action-state pairs

e.g., $S(\text{Arad}) = \{ \langle \text{Arad} \rightarrow \text{Zerind}, \text{Zerind} \rangle, \dots \}$

3. **goal test** (or set of goal states)

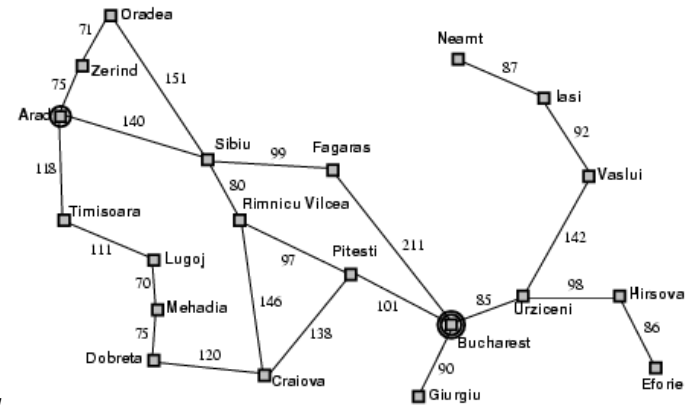
e.g., $x = \text{"at Bucharest"} , \text{Checkmate}(x)$

4. **path cost** (additive)

e.g., sum of distances, number of actions executed, etc.

$c(x,a,y)$ is the step cost, assumed to be ≥ 0

A **solution** is a sequence of actions leading from the initial state to a goal state



Example: Formulating the Navigation Problem

- Set of States
 - individual cities
 - e.g., Irvine, SF, Las Vegas, Reno, Boise, Phoenix, Denver
- Operators
 - freeway routes from one city to another
 - e.g., Irvine to SF via 5, SF to Seattle, etc
- Start State
 - current city where we are, Irvine
- Goal States
 - set of cities we would like to be in
 - e.g., cities which are closer than Irvine
- Solution
 - a specific goal city, e.g., Boise
 - a sequence of operators which get us there,
 - e.g., Irvine to SF via 5, SF to Reno via 80, etc

Abstraction

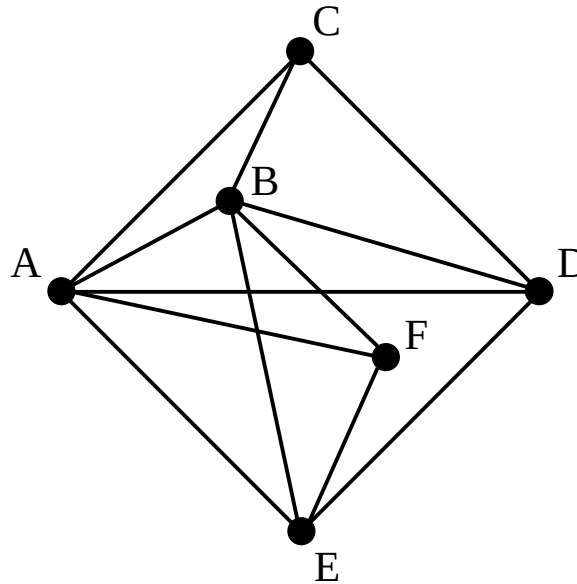
- Definition of Abstraction:
Process of removing irrelevant detail to create an abstract representation: ``high-level'', ignores irrelevant details
- Navigation Example: how do we define states and operators?
 - First step is to abstract "the big picture"
 - i.e., solve a map problem
 - nodes = cities, links = freeways/roads (a high-level description)
 - this description is an abstraction of the real problem
 - Can later worry about details like freeway onramps, refueling, etc
- Abstraction is critical for automated problem solving
 - must create an approximate, simplified, model of the world for the computer to deal with: real-world is too detailed to model exactly
 - good abstractions retain all important details

The State-Space Graph

- Graphs:
 - nodes, arcs, directed arcs, paths
- Search graphs:
 - States are nodes
 - operators are directed arcs
 - solution is a path from start S to goal G
- Problem formulation:
 - Give an abstract description of states, operators, initial state and goal state.
- Problem solving:
 - Generate a part of the search space that contains a solution

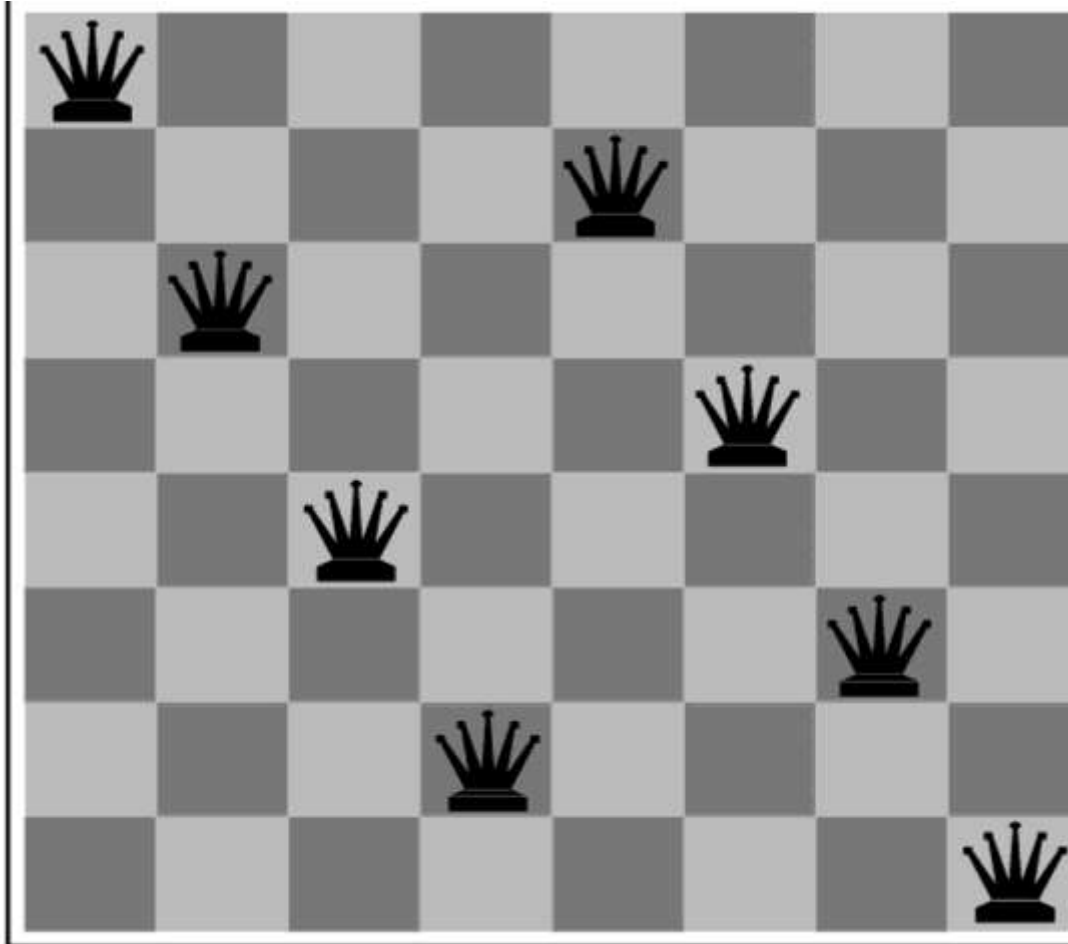
The Traveling Salesperson Problem

- Find the shortest tour that visits all cities without visiting any city twice and return to starting point.
- State: sequence of cities visited
- $S_0 = A$



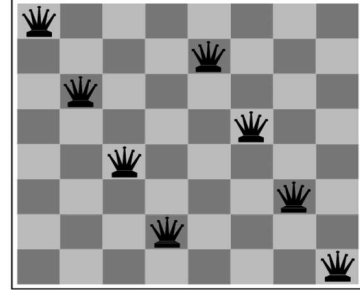
- $G =$ a complete tour

Example: 8-queens problem

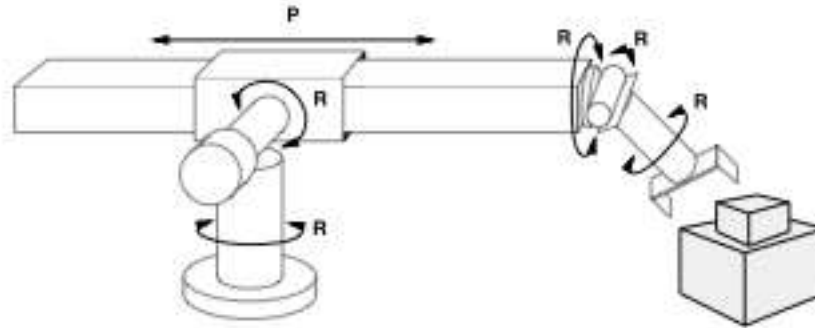


State-Space problem formulation

- states? -any arrangement of $n \leq 8$ queens
-or arrangements of $n \leq 8$ queens in leftmost n columns, 1 per column, such that no queen attacks any other.
- initial state? no queens on the board
- actions? -add queen to any empty square
-or add queen to leftmost empty square such that it is not attacked by other queens.
- goal test? 8 queens on the board, none attacked.
- path cost? 1 per move

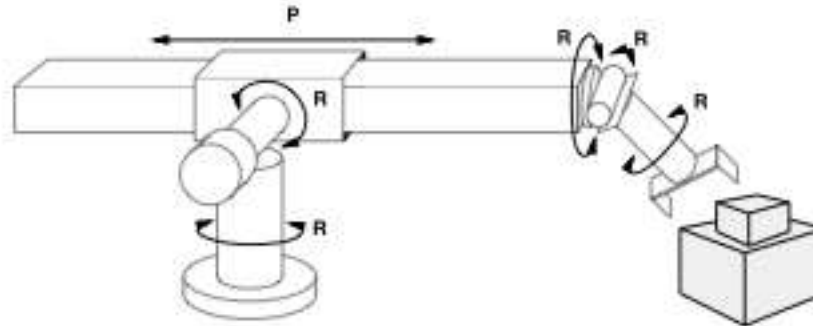


Example: Robot Assembly



- States
- Initial state
- Actions
- Goal test
- Path Cost

Example: Robot Assembly



- States: configuration of robot (angles, positions) and object parts
- Initial state: any configuration of robot and object parts
- Actions: continuous motion of robot joints
- Goal test: object assembled?
- Path Cost: time-taken or number of actions

Learning a spam email classifier

- States
- Initial state
- Actions
- Goal test
- Path Cost

Learning a spam email classifier

- States: settings of the parameters in our model
- Initial state: random parameter settings
- Actions: moving in parameter space
- Goal test: optimal accuracy on the training data
- Path Cost: time taken to find optimal parameters

(Note: this is an optimization problem – many machine learning problems can be cast as optimization)

Example: 8-puzzle

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

- states?
- initial state?
- actions?
- goal test?
- path cost?

Example: 8-puzzle

7	2	4
5		6
8	3	1

Start State

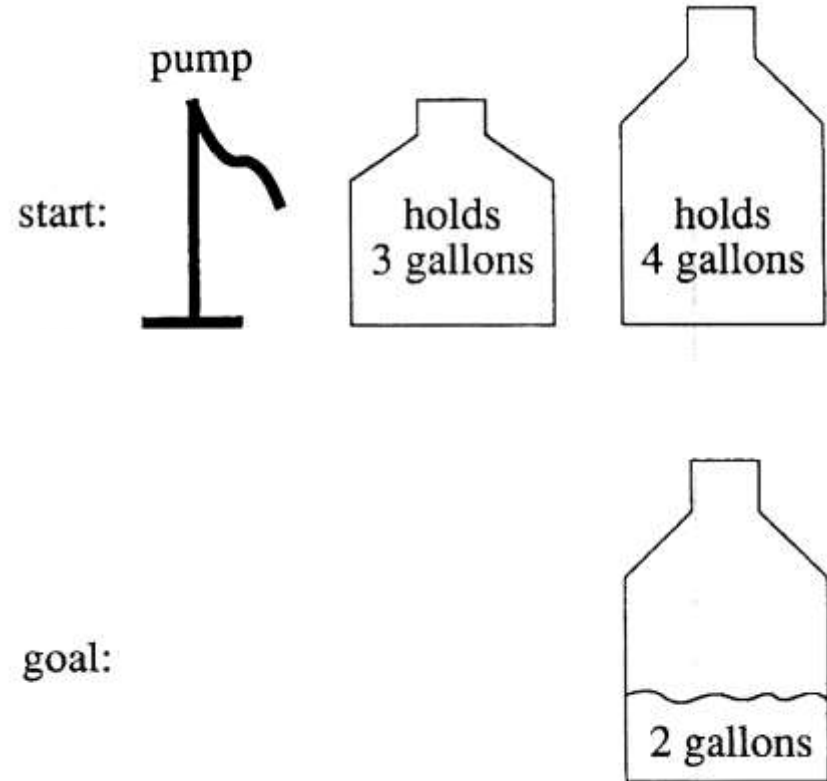
	1	2
3	4	5
6	7	8

Goal State

- states? locations of tiles
- initial state? given
- actions? move blank left, right, up, down
- goal test? goal state (given)
- path cost? 1 per move

A Water Jug Problem

- You have a 4-gallon and a 3-gallon water jug
- You have a faucet with an unlimited amount of water
- You need to get exactly 2 gallons in 4-gallon jug



Puzzle-solving as Search

- State representation: **(x, y)**
 - x: Contents of four gallon
 - y: Contents of three gallon
- Start state: **(0, 0)**
- Goal state **(2, n)**
- Operators
 - Fill 3-gallon from faucet, fill 4-gallon from faucet
 - Fill 3-gallon from 4-gallon , fill 4-gallon from 3-gallon
 - Empty 3-gallon into 4-gallon, empty 4-gallon into 3-gallon
 - Dump 3-gallon down drain, dump 4-gallon down drain

Production Rules for the Water Jug Problem

- | | |
|--|---|
| 1 $(x,y) \rightarrow (4,y)$
if $x < 4$ | Fill the 4-gallon jug |
| 2 $(x,y) \rightarrow (x,3)$
if $y < 3$ | Fill the 3-gallon jug |
| 3 $(x,y) \rightarrow (x - d,y)$
if $x > 0$ | Pour some water out of the 4-gallon jug |
| 4 $(x,y) \rightarrow (x,y - d)$
if $y > 0$ | Pour some water out of the 3-gallon jug |
| 5 $(x,y) \rightarrow (0,y)$
if $x > 0$ | Empty the 4-gallon jug on the ground |
| 6 $(x,y) \rightarrow (x,0)$
if $y > 0$ | Empty the 3-gallon jug on the ground |
| 7 $(x,y) \rightarrow (4,y - (4 - x))$
if $x + y \geq 4$ and $y > 0$ | Pour water from the 3-gallon jug into the 4-gallon jug until the 4-gallon jug is full |

The Water Jug Problem (cont'd)

8 $(x,y) \rightarrow (x - (3 - y), 3)$
if $x + y \geq 3$ and $x > 0$

Pour water from the 4-gallon jug into the 3-gallon jug until the 3-gallon jug is full

9 $(x,y) \rightarrow (x + y, 0)$
if $x + y \leq 4$ and $y > 0$

Pour all the water from the 3-gallon jug into the 4-gallon jug

10 $(x,y) \rightarrow (0, x + y)$
if $x + y \leq 3$ and $x > 0$

Pour all the water from the 4-gallon jug into the 3-gallon jug

One Solution to the Water Jug Problem

Gallons in the 4-Gallon Jug	Gallons in the 3-Gallon Jug	Rule Applied
0	0	2
0	3	9
3	0	2
3	3	7
4	2	5
0	2	9
2	0	

VLSI Layout Problem

- Require positioning millions of components and connections on a chip to **minimize area, minimize circuit delays, minimize stray capacitances, and maximize manufacturing yield**
- The layout problem comes after the logical design phase and is usually split into two parts: **cell layout** and **channel routing**
- In cell layout, the primitive components of the circuit are grouped into cells, each of which performs some recognized function

VLSI Layout Problem (cont'd)

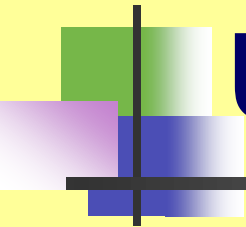
- Each cell has a fixed footprint (size and shape) and requires a certain number of connections to each of the other cells
- The aim is to place the cells on the chip so that they do not overlap and so that there is room for the connecting wires to be placed between the cells
- Channel routing finds a specific route for each wire through the gaps between the cells. These search problems are extremely complex, but definitely worth solving.

Next Topics

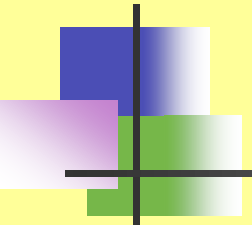
- Uninformed search
 - Breadth-first, depth-first
 - Uniform cost
 - Iterative deepening
- Informed (heuristic) search
 - Greedy best-first
 - A*
 - Memory-bounded heuristic search
 - And more....
- Local search and optimization
 - Hill-climbing
 - Simulated annealing
 - Genetic algorithms

Summary

- Problem-solving agents where search consists of
 - state space
 - operators
 - start state
 - goal states
- Abstraction and problem formulation



Uninformed Search



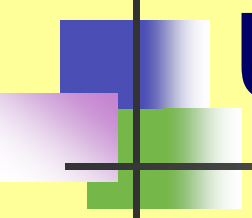
Measuring problem-solving performance

Completeness: Is the algorithm guaranteed to find a solution when there is one?

Optimality: Does the strategy find the optimal solution, as defined on page 68?

Time complexity: How long does it take to find a solution?

Space complexity: How much memory is needed to perform the search?



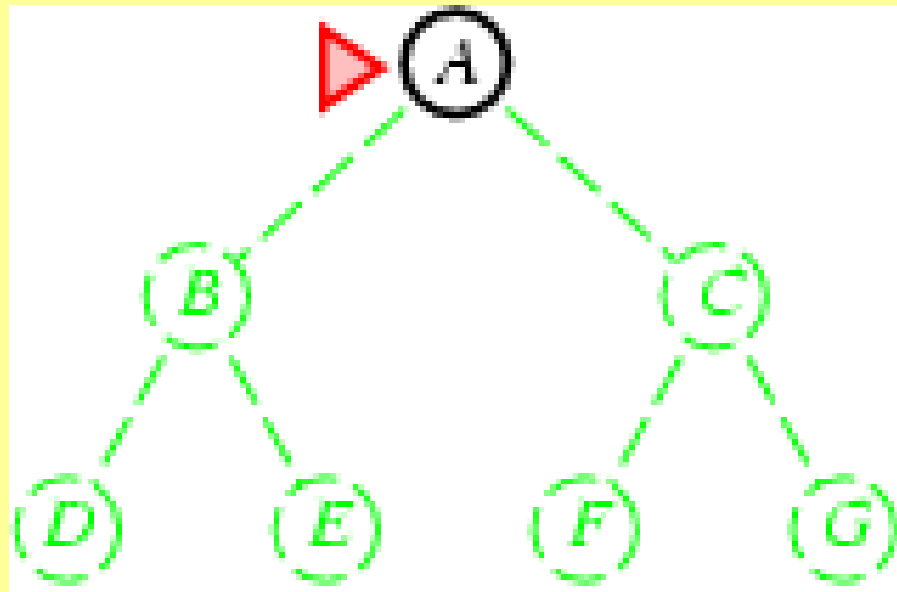
Uninformed search strategies

- **Uninformed:** While searching you have no clue whether one non-goal state is better than any other. Your search is blind.
- **Various blind strategies:**
 - Breadth-first search
 - Uniform-cost search
 - Depth-first search
 - Iterative deepening search

Breadth-first search

- Expand shallowest unexpanded node
- Implementation:
 - *fringe* is a first-in-first-out (FIFO) queue, i.e., new successors go at end of the queue.

Is A a goal state?

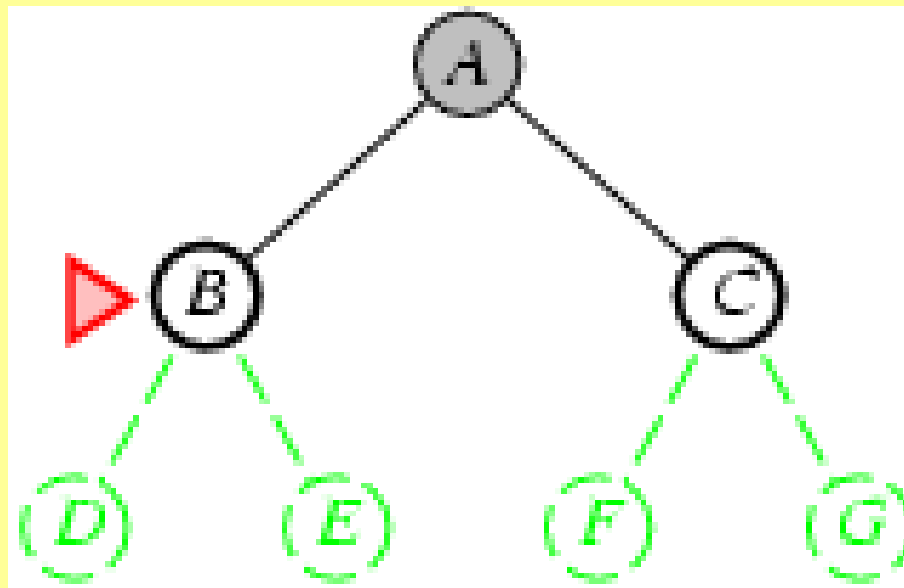


Breadth-first search

- Expand shallowest unexpanded node
- Implementation:
 - *fringe* is a FIFO queue, i.e., new successors go at end

Expand:
fringe = [B,C]

Is B a goal state?

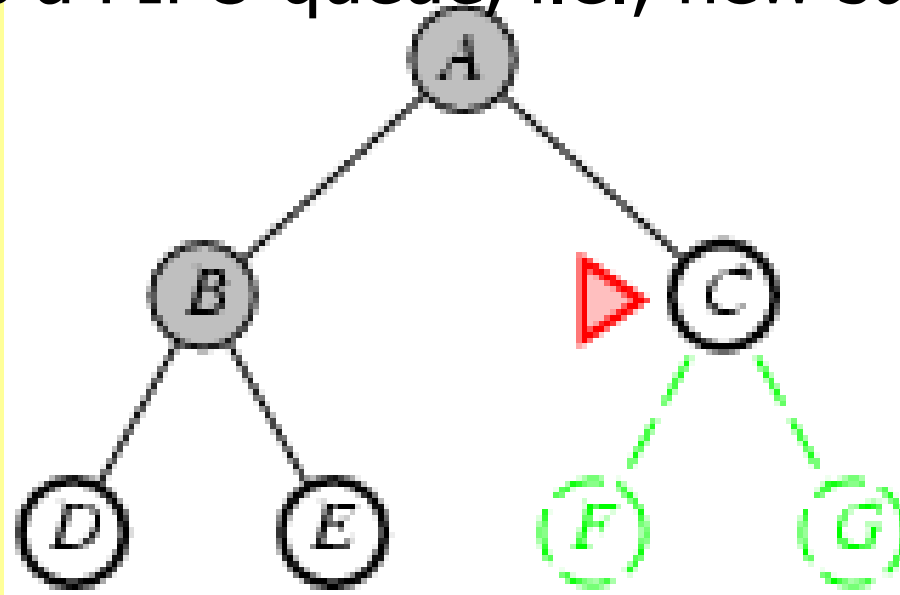


Breadth-first search

- Expand shallowest unexpanded node
-
- Implementation:
 - *fringe* is a FIFO queue, i.e., new successors go at end

Expand:
fringe=[C,D,E]

Is C a goal state?

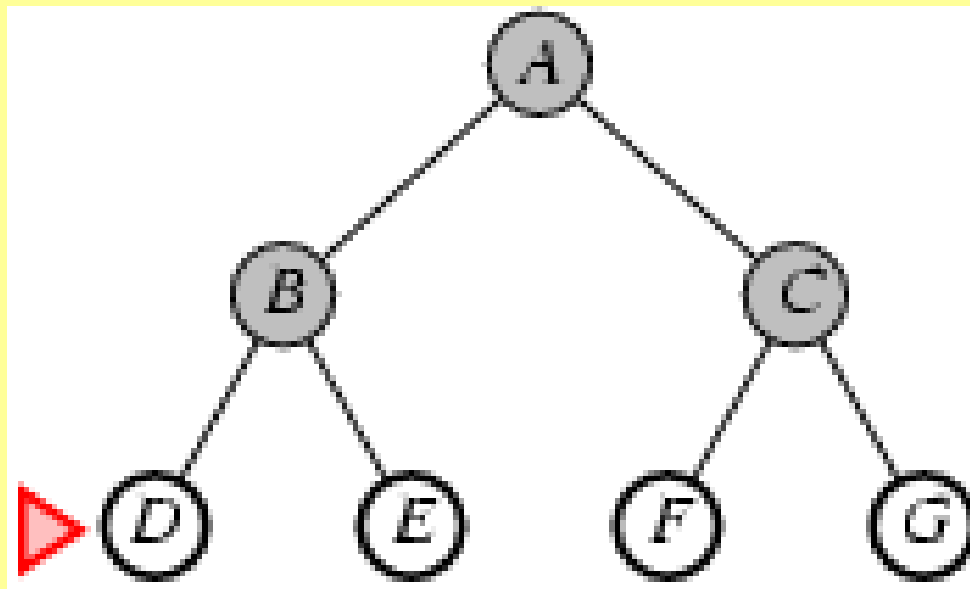


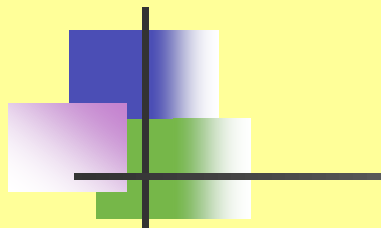
Breadth-first search

- Expand shallowest unexpanded node
- Implementation:
 - *fringe* is a FIFO queue, i.e., new successors go at end

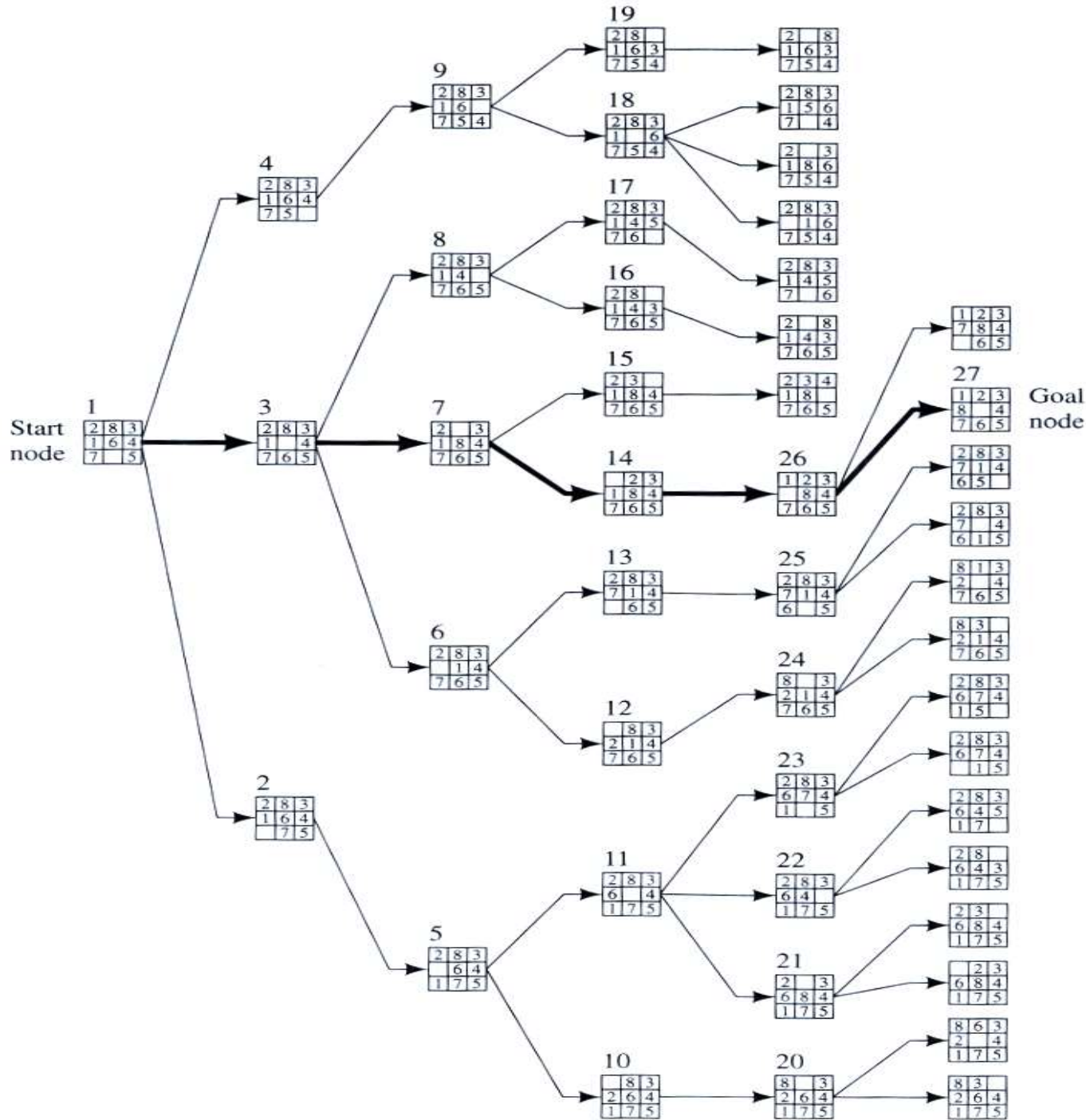
Expand:
fringe=[D,E,F,G]

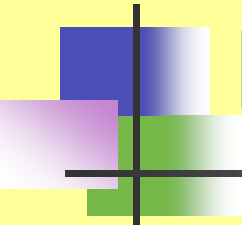
Is D a goal state?





Example BFS





Properties of breadth-first search

- Complete? Yes it always reaches goal (if b is finite)
- Time? $1+b+b^2+b^3+\dots +b^d + (b^{d+1}-b)) = O(b^{d+1})$
(this is the number of nodes we generate)
- Space? $O(b^{d+1})$ (keeps every node in memory,
either in fringe or on a path to fringe).
- Optimal? Yes (if we guarantee that deeper
solutions are less optimal, e.g. step-cost=1).
- **Space** is the bigger problem (more than time)

Uniform-cost search

Breadth-first is only optimal if step costs is increasing with depth (e.g. constant). Can we guarantee optimality for any step cost?

Uniform-cost Search: Expand node with smallest path cost $g(n)$.

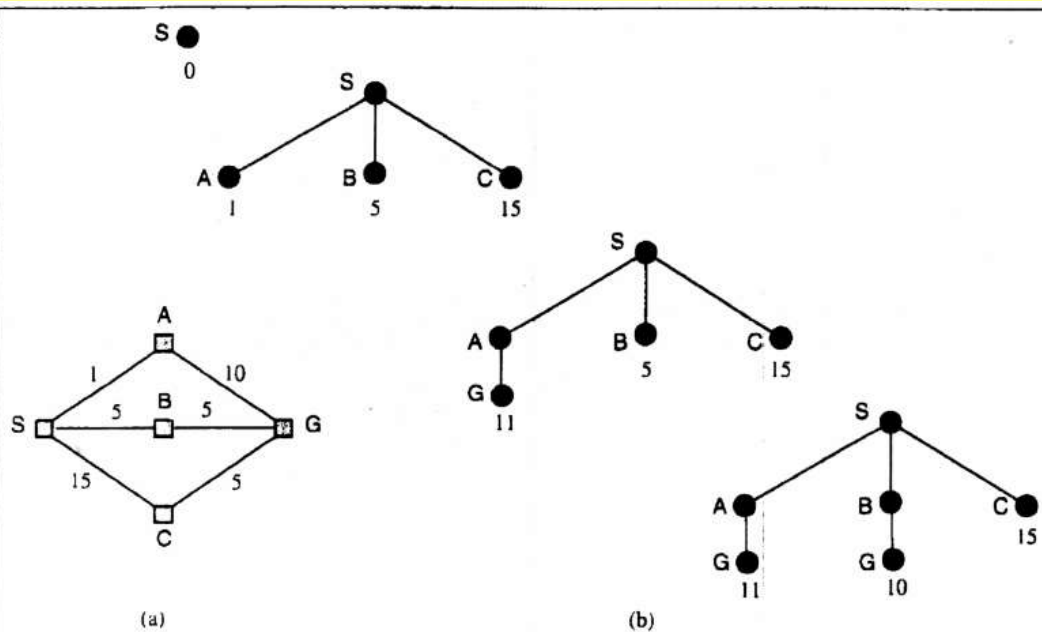
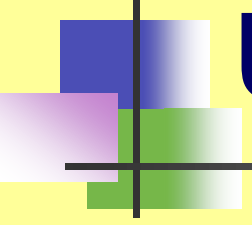


Figure 3.13 A route-finding problem. (a) The state space, showing the cost for each operator. (b) Progression of the search. Each node is labelled with $g(n)$. At the next step, the goal node with $g = 10$ will be selected.



Uniform-cost search

Implementation: *fringe* = queue ordered by path cost
Equivalent to breadth-first if all step costs all equal.

Complete? Yes, if step cost $\geq \epsilon$
(otherwise it can get stuck in infinite loops)

Time? # of nodes with *path cost* $\leq$ cost of optimal solution.

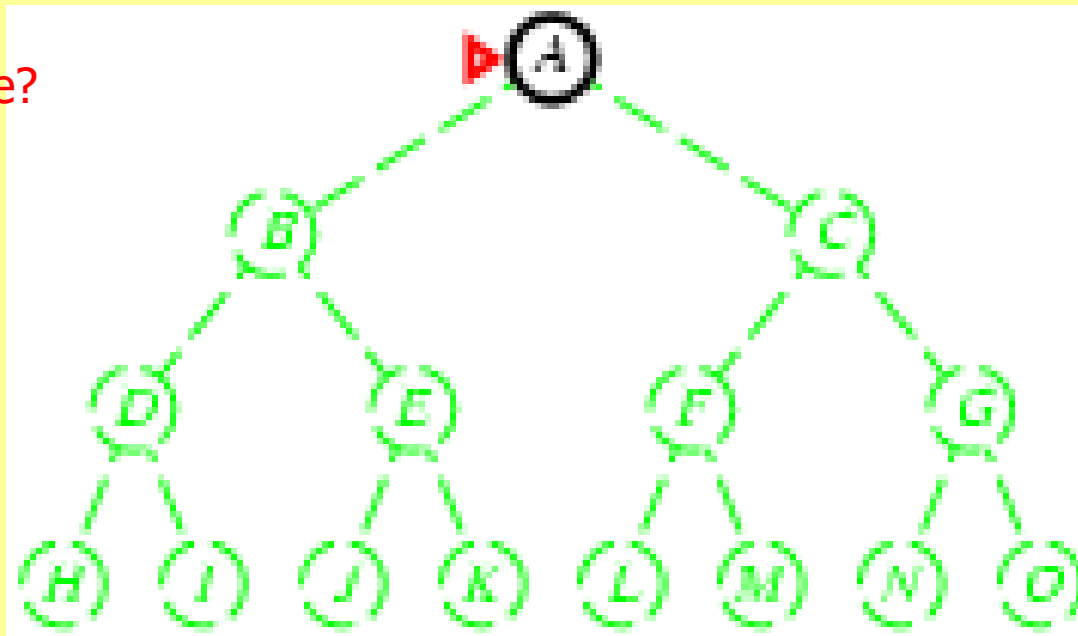
Space? # of nodes on paths with path cost $\leq$ cost of optimal solution.

Optimal? Yes, for any step cost.

Depth-first search

- Expand *deepest* unexpanded node
- Implementation:
 - *fringe* = Last In First Out (LIPO) queue, i.e., put successors at front

Is A a goal state?



Depth-first search

- Expand deepest unexpanded node



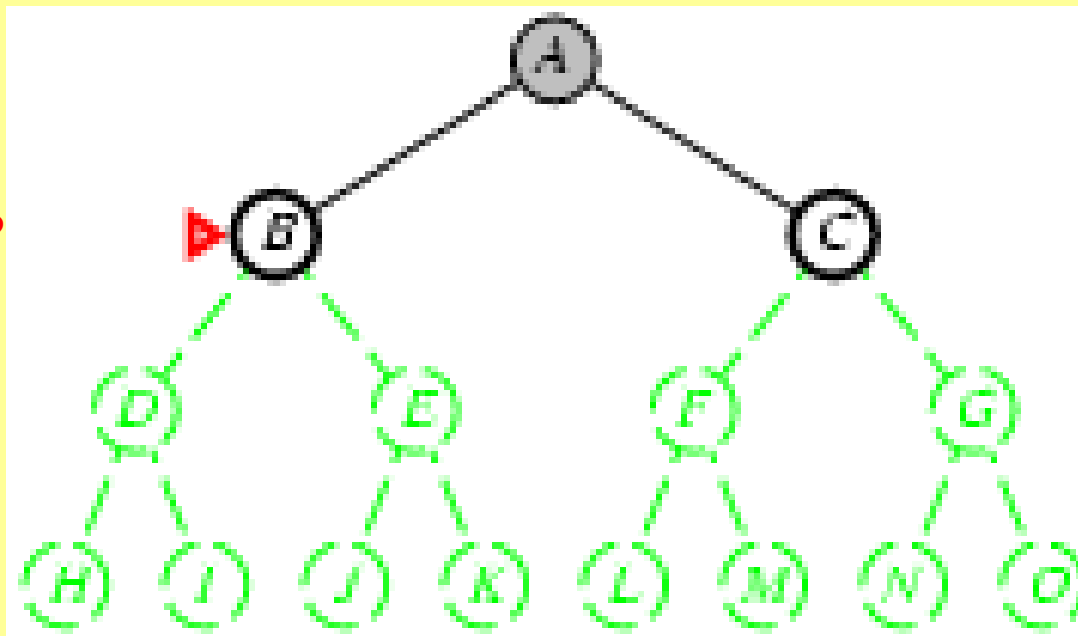
- Implementation:

- *fringe*

queue=[B,C]



Is B a goal state?



front

Depth-first search

- Expand deepest unexpanded node



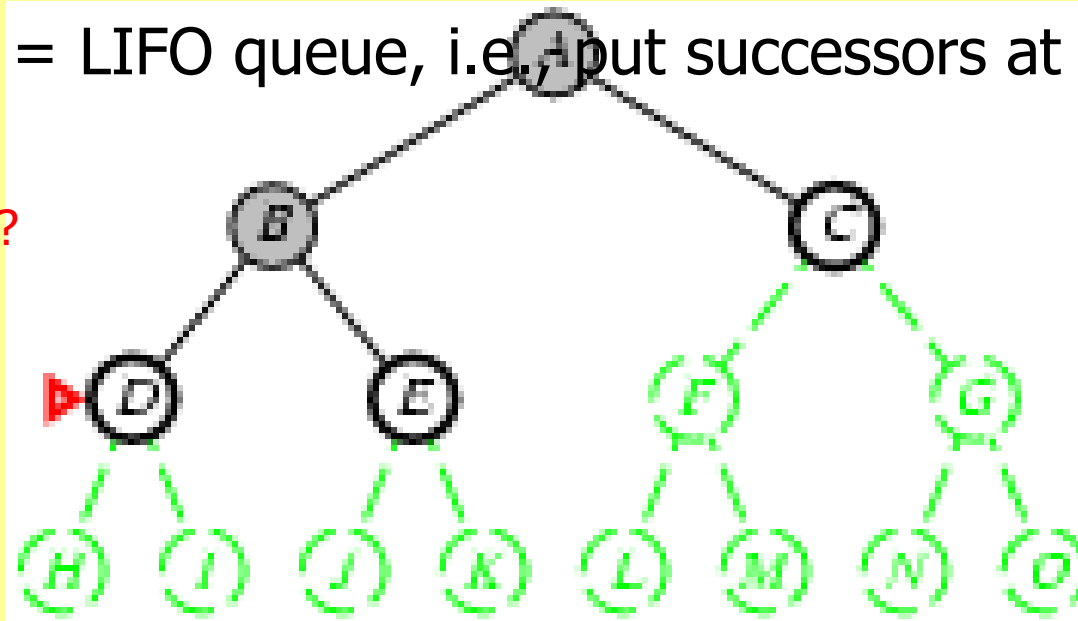
- Implementation:

- *fringe* = LIFO queue, i.e., put successors at front

queue=[D,E,C]



Is D = goal state?



Depth-first search

- Expand deepest unexpanded node

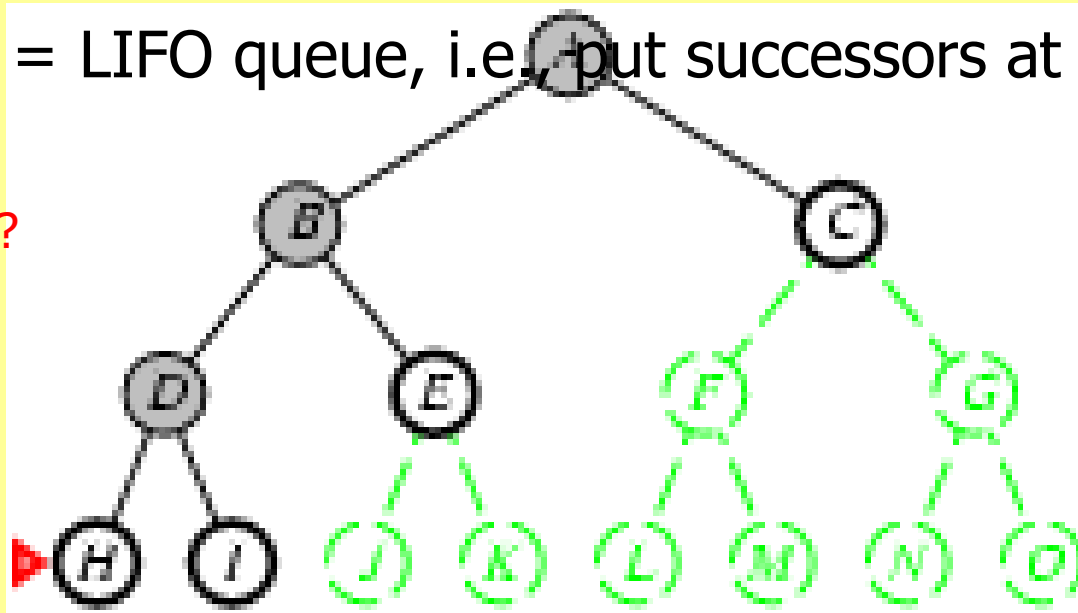


- Implementation:

- *fringe* = LIFO queue, i.e., put successors at front

queue=[H,I,E,C]

Is H = goal state?



Depth-first search

- Expand deepest unexpanded node

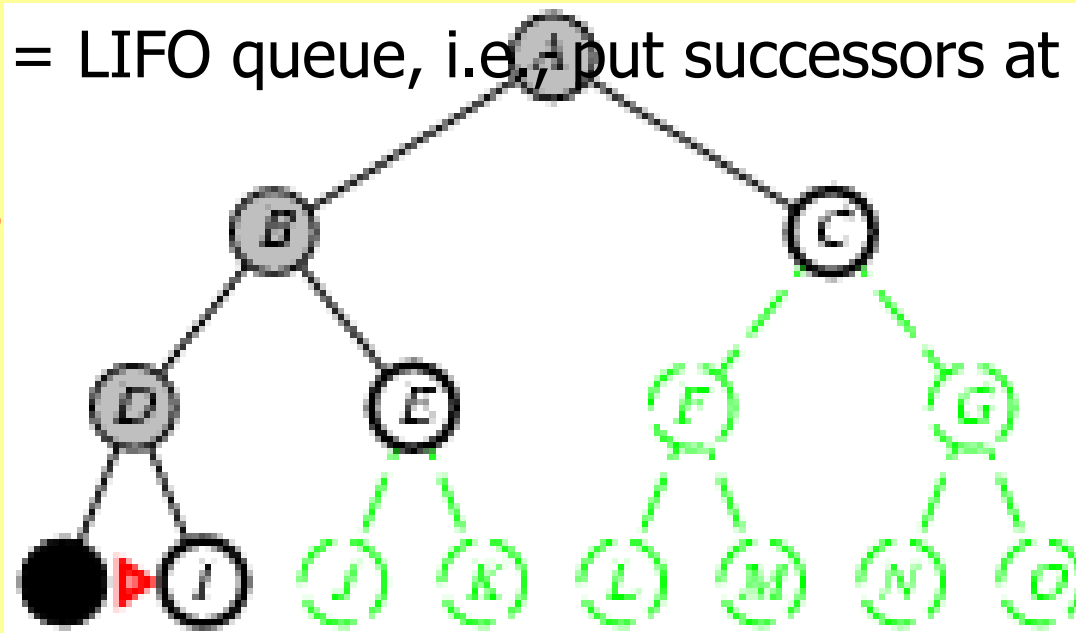


- Implementation:

- *fringe* = LIFO queue, i.e., put successors at front

queue=[I,E,C]

Is I = goal state?



Depth-first search

- Expand deepest unexpanded node



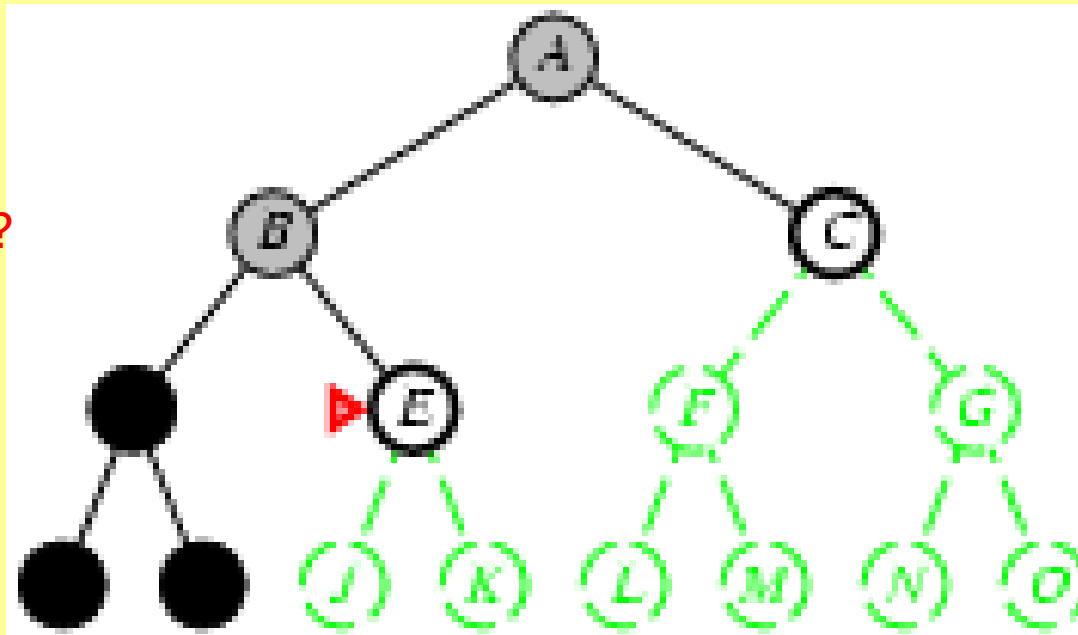
- Implementation:

- *fringe*

queue=[E,C]



Is E = goal state?



front

Depth-first search

- Expand deepest unexpanded node



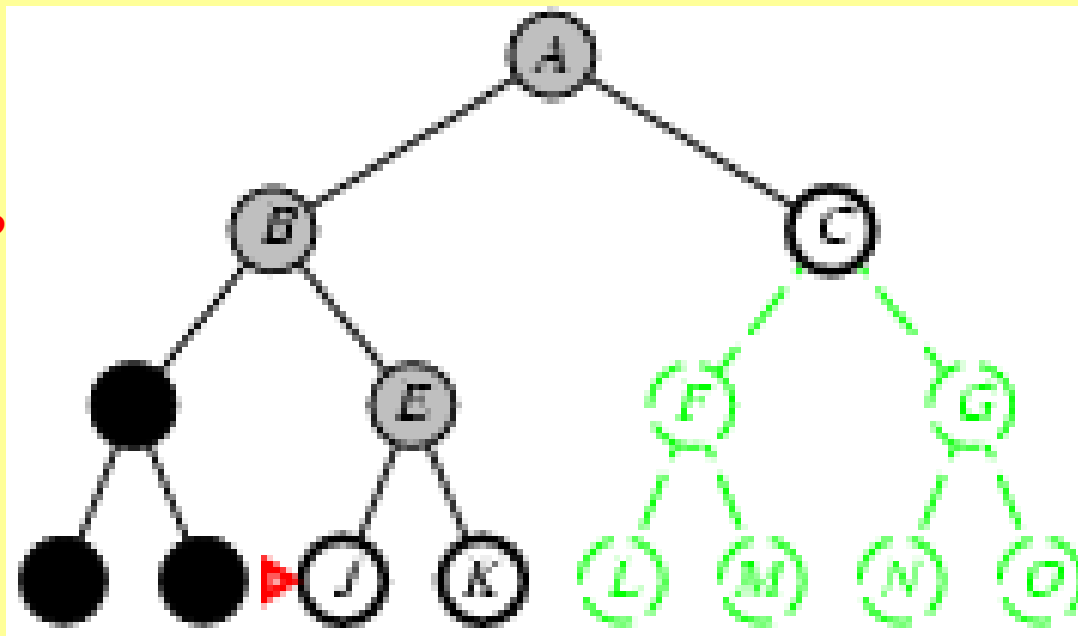
- Implementation:

- *fringe*

- *queue=[J,K,C]*



- Is J = goal state?



front

Depth-first search

- Expand deepest unexpanded node



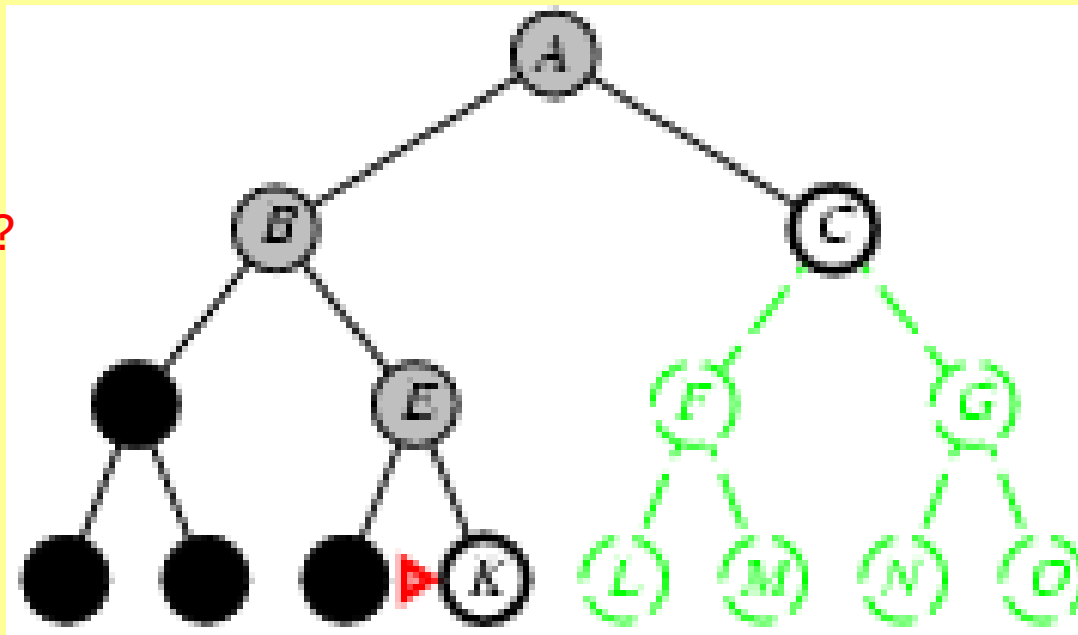
- Implementation:

- *fringe*

queue=[K,C]



Is K = goal state?



front

Depth-first search

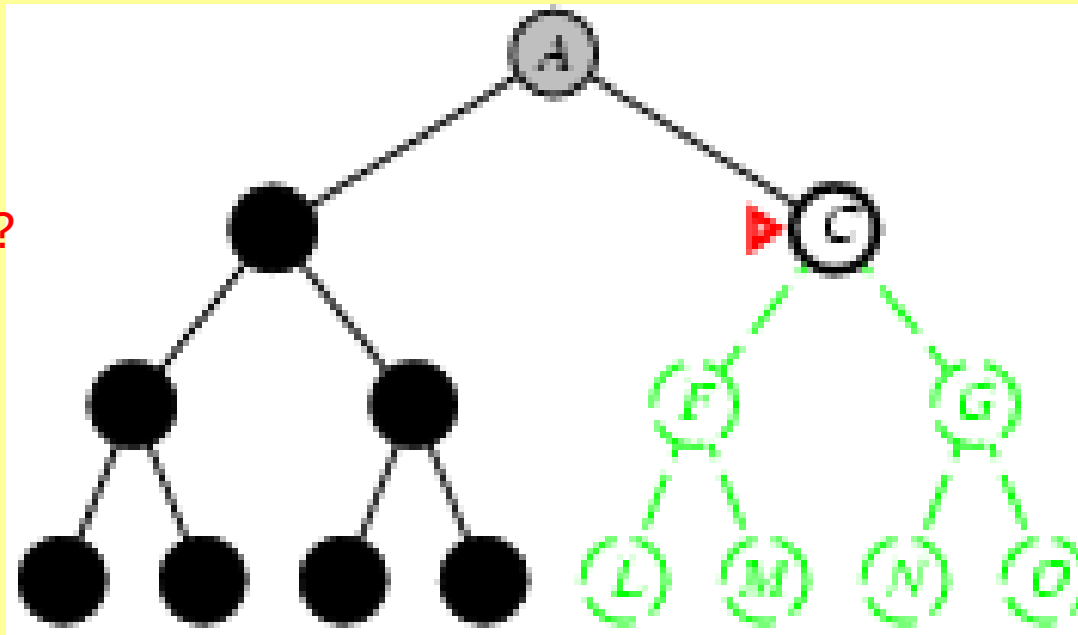
- Expand deepest unexpanded node

- Implementation:

- *fringe*

- *queue*=[C]

- Is C = goal state?



front

Depth-first search

- Expand deepest unexpanded node



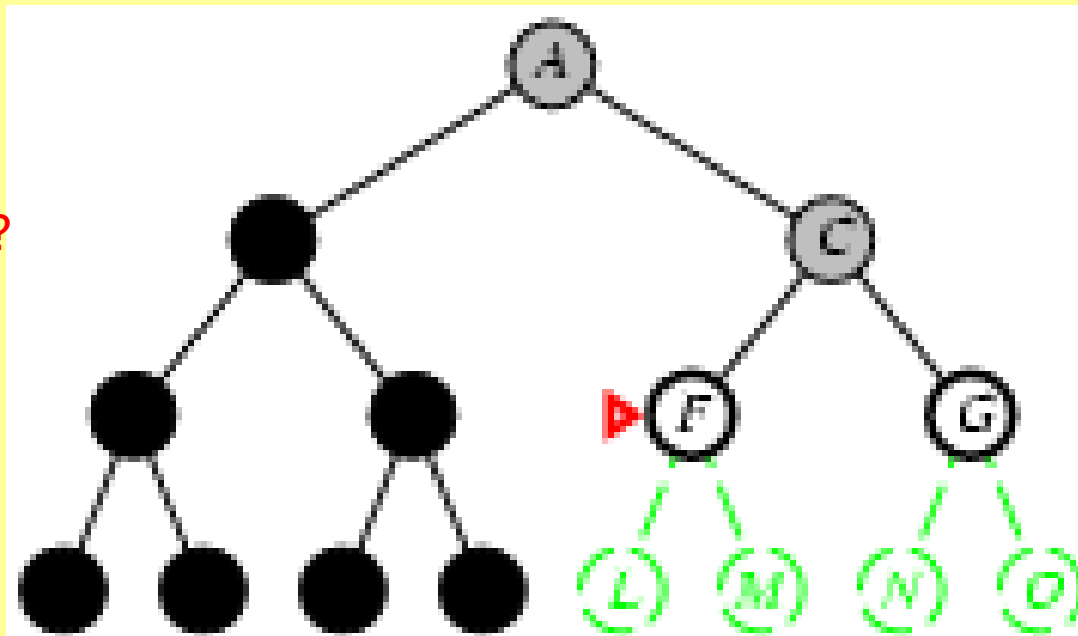
- Implementation:

- *fringe*

queue=[F,G]



Is F = goal state?



Depth-first search

- Expand deepest unexpanded node



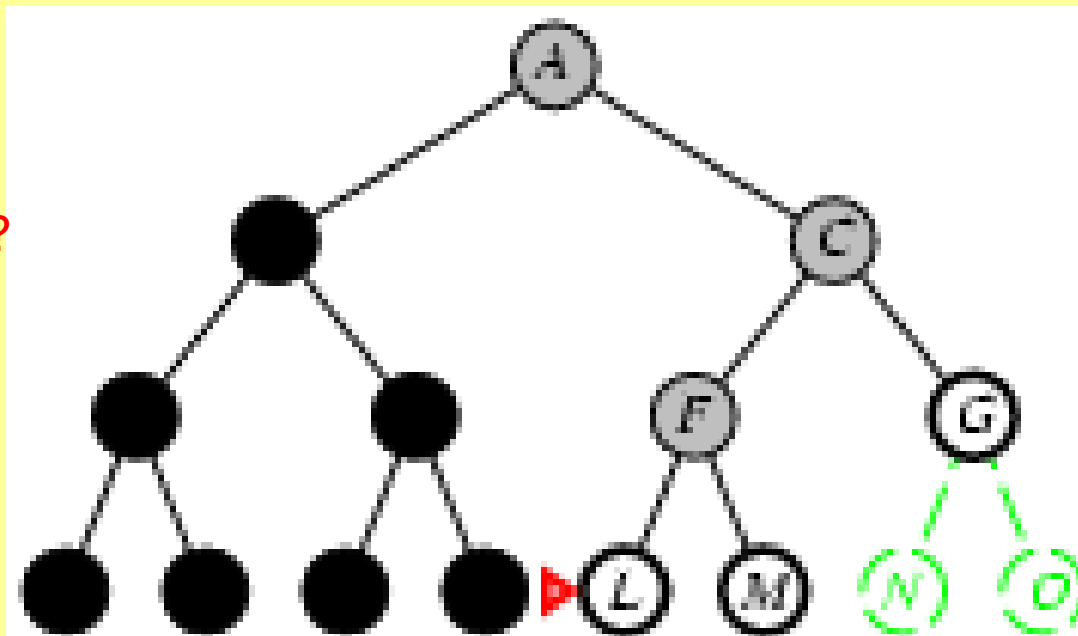
- Implementation:

- *fringe*

- *queue*=[L,M,G]



- Is L = goal state?



front

Depth-first search

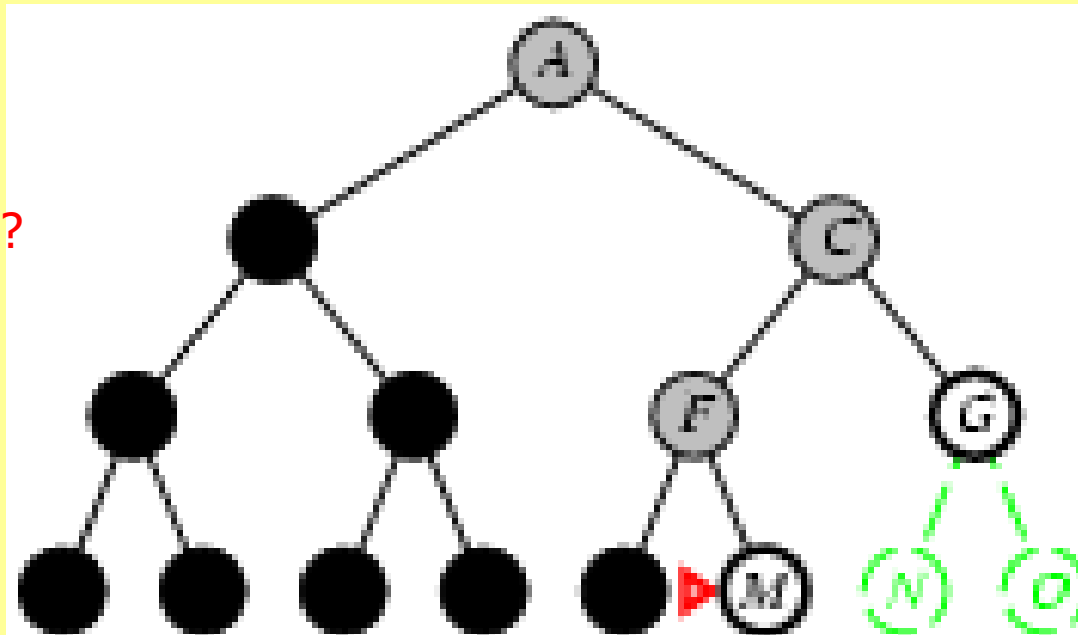
- Expand deepest unexpanded node

- Implementation:

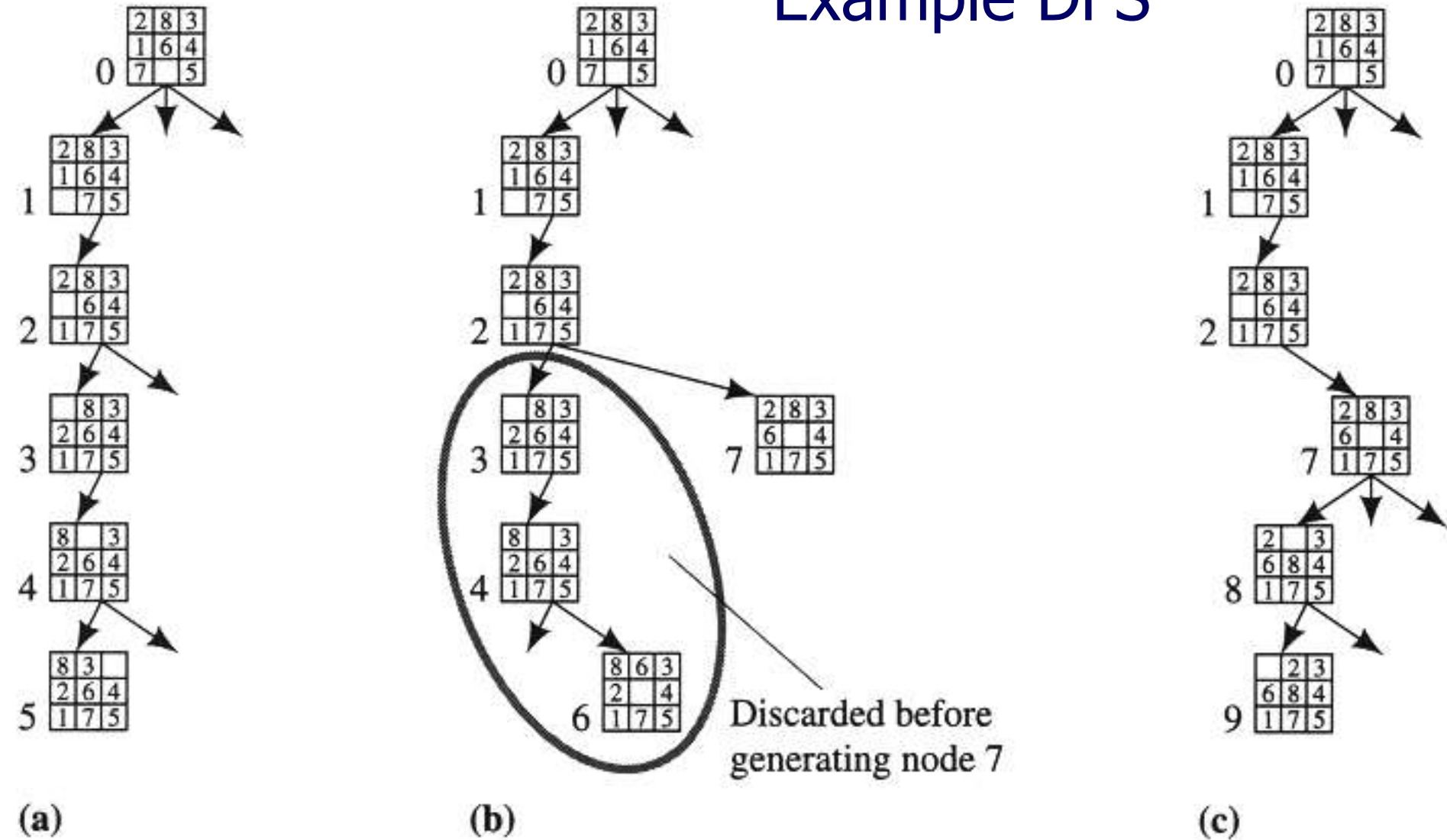
- *fringe*

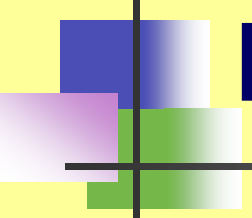
- *queue*=[M,G]

- Is M = goal state?



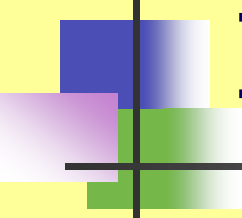
Example DFS





Properties of depth-first search

- Complete? No: fails in infinite-depth spaces
Can modify to avoid repeated states along path
- Time? $O(b^m)$ with m =maximum depth
- terrible if m is much larger than d
 - but if solutions are dense, may be much faster than breadth-first
- Space? $O(bm)$, i.e., linear space! (we only need to remember a single path + expanded unexplored nodes)
- Optimal? No (It may find a non-optimal goal first)



Iterative deepening search

- To avoid the infinite depth problem of DFS, we can decide to only search until depth L , i.e. we don't expand beyond depth L .
→ Depth-Limited Search
- What of solution is deeper than L ? → Increase L iteratively.
→ Iterative Deepening Search
- As we shall see: this inherits the memory advantage of Depth-First search.

Iterative deepening search $L=0$

Limit = 0



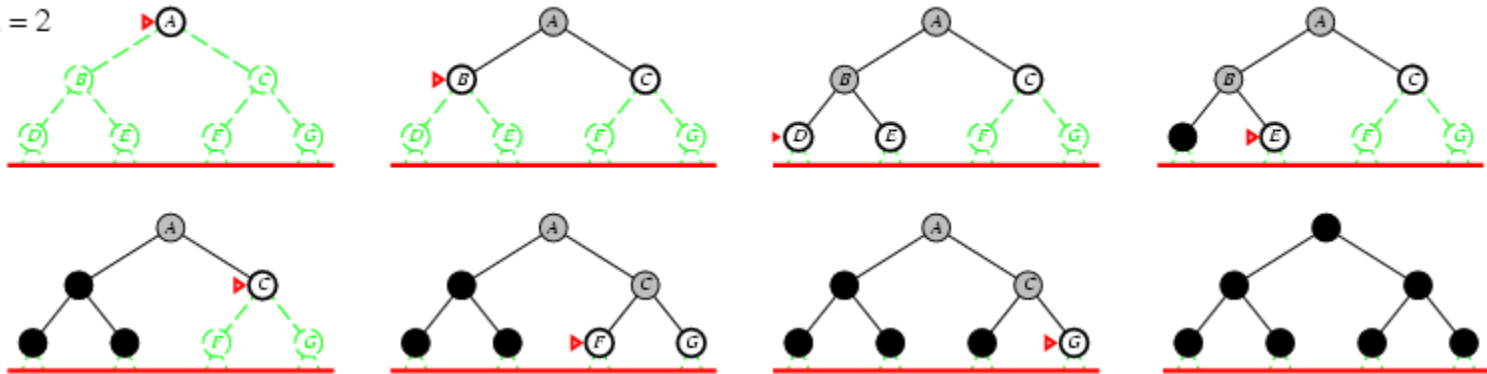
Iterative deepening search $L=1$

Limit = 1



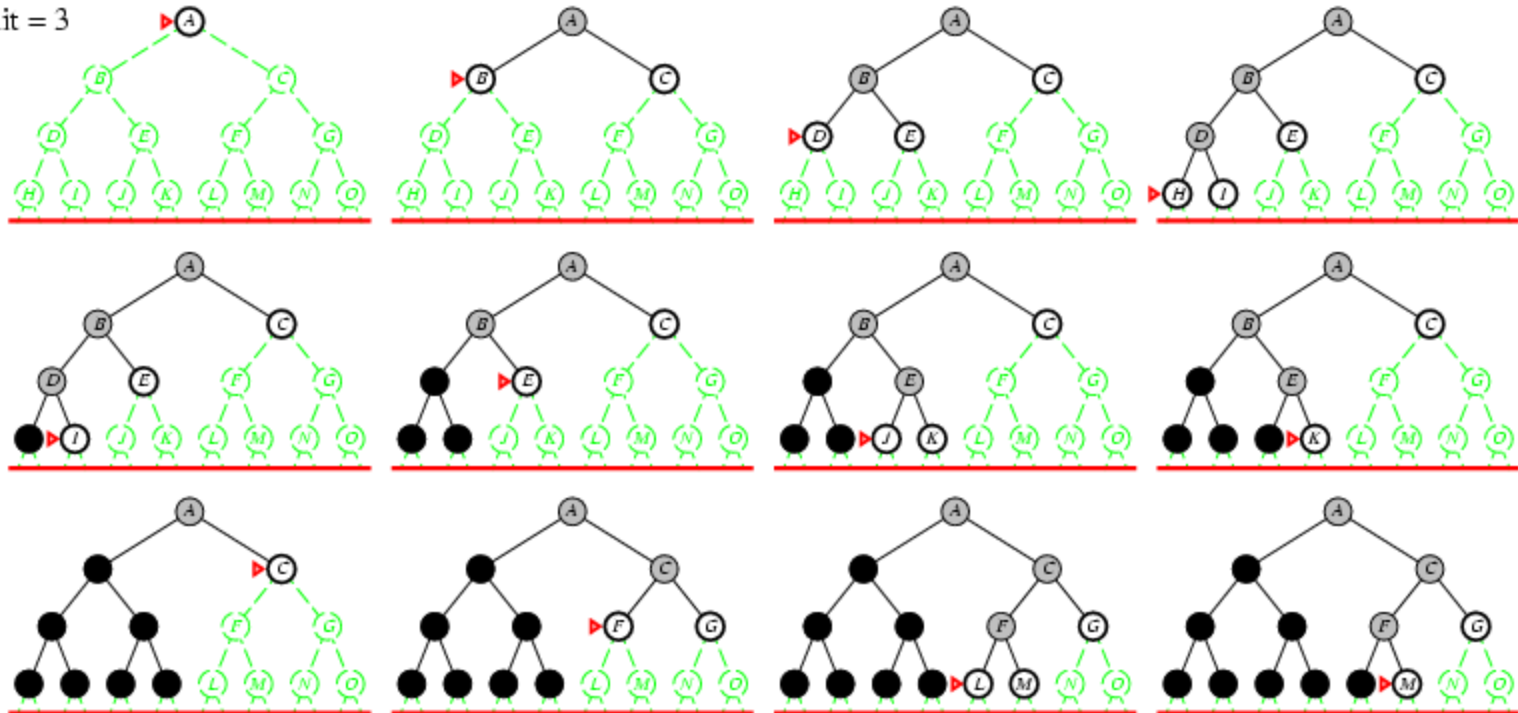
Iterative deepening search $L=2$

Limit = 2



Iterative deepening search $L=3$

Limit = 3



Iterative deepening search

- Number of nodes generated in a depth-limited search to depth d with branching factor b :

$$N_{DLS} = b^0 + b^1 + b^2 + \dots + b^{d-2} + b^{d-1} + b^d$$

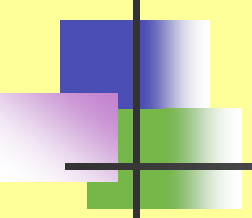
- Number of nodes generated in an iterative deepening search to depth d with branching factor b :

$$N_{IDS} = (d+1)b^0 + d b^1 + (d-1)b^2 + \dots + 3b^{d-2} + 2b^{d-1} + 1b^d = O(b^d) \neq O(b^{d+1})$$

- For $b = 10, d = 5$,

- $N_{DLS} = 1 + 10 + 100 + 1,000 + 10,000 + 100,000 = 111,111$
- $N_{IDS} = 6 + 50 + 400 + 3,000 + 20,000 + 100,000 = 123,450$
- $N_{BFS} = \dots = 1,111,100$

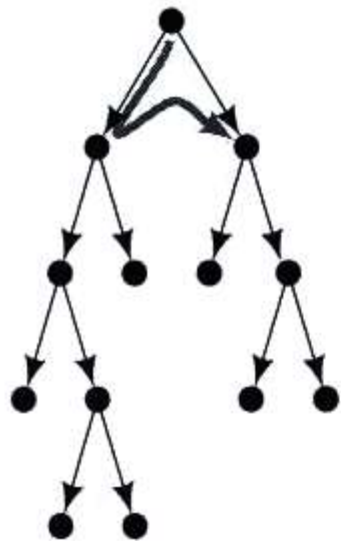
BFS



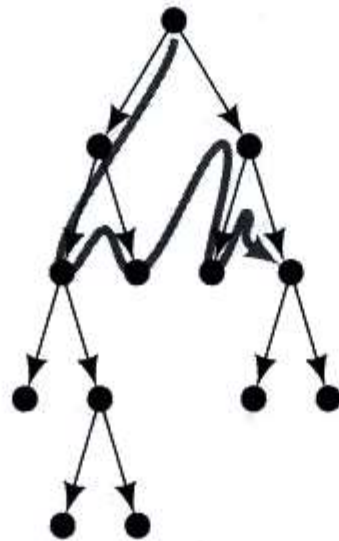
Properties of iterative deepening search

- Complete? Yes
- Time? $(d+1)b^0 + d b^1 + (d-1)b^2 + \dots + b^d = O(b^d)$
- Space? $O(bd)$
- Optimal? Yes, if step cost = 1 or increasing function of depth.

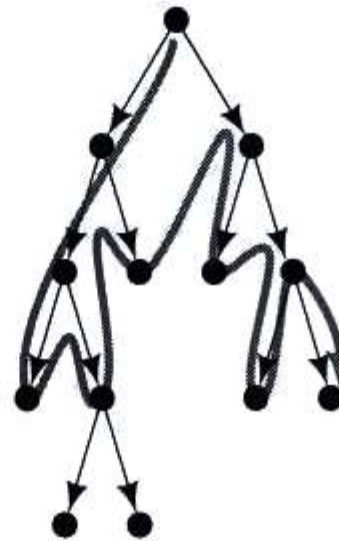
Example IDS



Depth bound = 1



Depth bound = 2

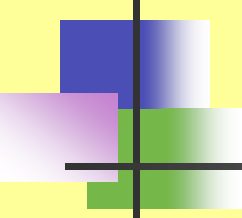


Depth bound = 3



Depth bound = 4

Stages in Iterative-Deepening Search



Bidirectional Search

■ Idea

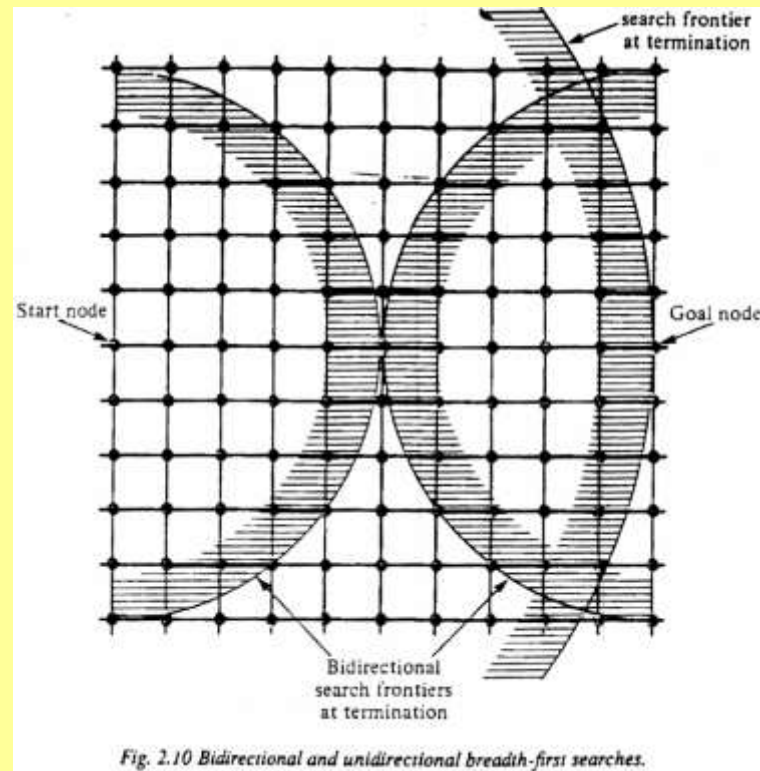
- simultaneously search forward from S and backwards from G
- stop when both “meet in the middle”
- need to keep track of the intersection of 2 open sets of nodes

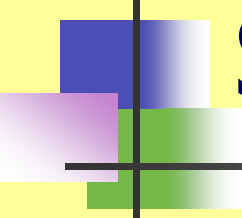
■ What does searching backwards from G mean

- need a way to specify the predecessors of G
 - this can be difficult,
 - e.g., predecessors of checkmate in chess?
- what if there are multiple goal states?
- what if there is only a goal test, no explicit list?

Bi-Directional Search

Complexity: time and space complexity are: $O(b^{d/2})$



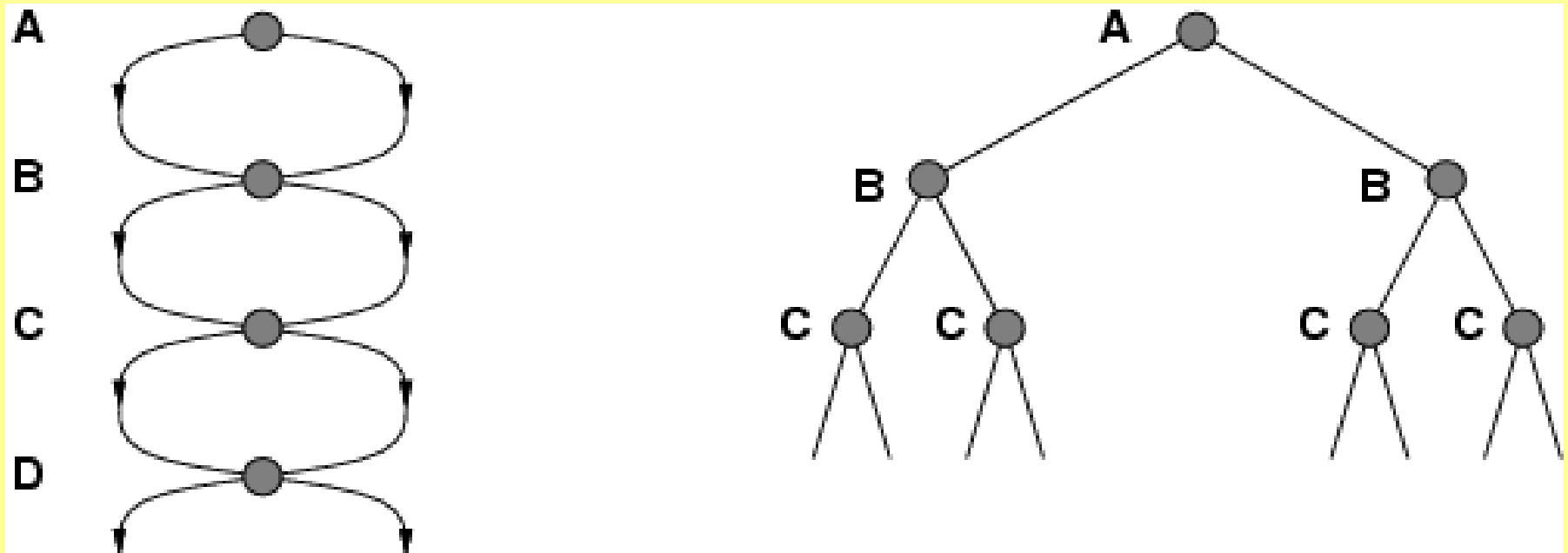


Summary of algorithms

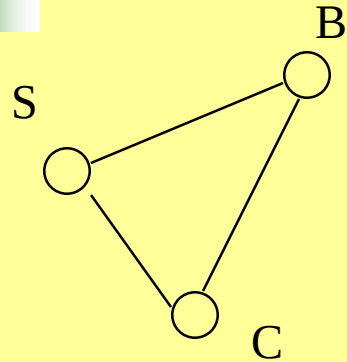
Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening
Complete?	Yes	Yes	No	No	Yes
Time	$O(b^{d+1})$	$O(b^{\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^l)$	$O(b^d)$
Space	$O(b^{d+1})$	$O(b^{\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(bl)$	$O(bd)$
Optimal?	Yes	Yes	No	No	Yes

Repeated states

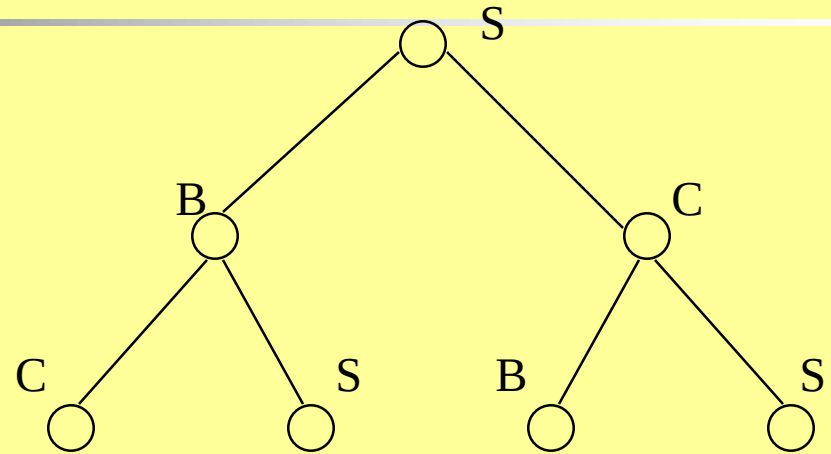
- Failure to detect repeated states can turn a linear problem into an exponential one!



Solutions to Repeated States

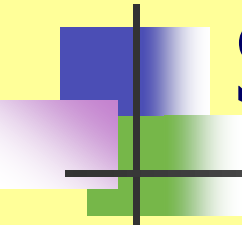


State Space



Example of a Search Tree

- **Method 1** ← suboptimal but practical
 - do not create paths containing cycles (loops)
- **Method 2** ← optimal but memory inefficient
 - never generate a state generated before
 - must keep track of all possible states (uses a lot of memory)
 - e.g., 8-puzzle problem, we have $9! = 362,880$ states



Summary

- Problem formulation usually requires abstracting away real-world details to define a state space that can feasibly be explored
- Variety of uninformed search strategies
- Iterative deepening search uses only linear space and not much more time than other uninformed algorithms

<http://www.cs.rmit.edu.au/AI-Search/Product/>
<http://aima.cs.berkeley.edu/demos.html> (for more demos)